# My Tcl Scripting and EDA Journey

## From Core Tcl to Practical HDL Automation

**By - Bibhu Asish Panda**

GitHub Profile | LinkedIn Profile

Learning Notes and Practical Exercises

# Contents


**1 Core Tcl Syntax** **1**

1.1 Introduction . . . 1
1.2 Learning Objectives . . . 1
1.3 Repository Exercises . . . 1
1.4 Variables and Basic Output . . . 1
1.4.1 Complete source: `01_variables_and_output.tcl` . . . 2
1.4.2 Expected output . . . 2
1.4.3 Command reference . . . 2
1.5 Expressions and Arithmetic . . . 2
1.5.1 Complete source: `02_expressions.tcl` . . . 2
1.5.2 Expected output . . . 3
1.5.3 Arithmetic operators . . . 3
1.6 Conditional Statements . . . 3
1.6.1 Complete source: `03_conditions.tcl` . . . 3
1.6.2 Expected output . . . 4
1.6.3 Comparison operators . . . 4
1.7 Loops . . . 4
1.7.1 Complete source: `04_loops.tcl` . . . 4
1.7.2 Expected output . . . 5
1.7.3 Loop structure . . . 5
1.7.4 Loop command reference . . . 5
1.8 Basic Tcl Lists . . . 6
1.8.1 Complete source: `05_lists.tcl` . . . 6
1.8.2 Expected output . . . 6
1.8.3 List command reference . . . 6
1.9 Tcl Substitution and Grouping . . . 7
1.9.1 Examples . . . 7
1.9.2 Expected output . . . 7
1.10 Key Concepts to Remember . . . 7
1.11 Review Exercises . . . 8
1.12 Phase 01 Summary . . . 8

**2 Data Handling** **9**

2.1 Introduction . . . 9
2.2 Learning Objectives . . . 9
2.3 Repository Exercises . . . 9
2.4 Lists and Advanced List Operations . . . 9

2.4.1 Complete source: `06_list_operations.tcl` . . . 9
2.4.2 Command reference . . . 10
2.4.3 Expected output . . . 10
2.4.4 How the list changes . . . 11
2.5 Tcl Arrays . . . 11
2.5.1 Complete source: `07_arrays.tcl` . . . 11
2.5.2 Key commands . . . 11
2.5.3 Expected output . . . 12
2.6 Dictionaries . . . 12
2.6.1 Complete source: `08_dictionaries.tcl` . . . 12
2.6.2 Key commands . . . 12
2.6.3 Expected output . . . 13
2.7 Nested Data . . . 13
2.7.1 Complete source: `09_nested_data.tcl` . . . 13
2.7.2 Expected output . . . 13
2.7.3 Understanding the nested structure . . . 14
2.8 Choosing Between Lists, Arrays, and Dictionaries . . . 14
2.9 Practical Automation Uses . . . 14
2.9.1 Lists . . . 14
2.9.2 Arrays . . . 14
2.9.3 Dictionaries . . . 14
2.9.4 Nested structures . . . 15
2.10 Review Exercises . . . 15
2.11 Important Commands at a Glance . . . 15
2.12 Chapter Summary . . . 15

**3 Procedures** **17**
3.1 Introduction . . . 17
3.2 Learning Objectives . . . 17
3.3 Repository Files . . . 17
3.4 Procedure Basics . . . 18
3.4.1 What is a procedure? . . . 18
3.4.2 Repository example: basic procedure . . . 18
3.5 Procedure Arguments . . . 18
3.5.1 Parameters and arguments . . . 18
3.5.2 Repository example: passing arguments . . . 18
3.6 Returning Values . . . 19
3.6.1 The `return` command . . . 19
3.6.2 Repository example: return a calculation . . . 19
3.6.3 Additional example from the phase README . . . 20
3.7 Variable Scope . . . 20
3.7.1 Local variables . . . 20
3.7.2 Repository example: local and global names . . . 20
3.7.3 Accessing a global variable . . . 20
3.8 Procedure Structure and Reuse . . . 21
3.9 Important Commands Reference . . . 21

3.10 Practical Applications in Automation . . . 22
3.11 Review Exercises . . . 22
3.12 Phase Summary . . . 22

**4 String Processing 23**
4.1 Introduction . . . 23
4.2 Learning Objectives . . . 23
4.3 14 — Basic String Operations . . . 23
4.3.1 Concept . . . 23
4.3.2 Complete Source: `14_string_operations.tcl` . . . 23
4.3.3 Expected Output . . . 24
4.4 15 — String Formatting . . . 24
4.4.1 Concept . . . 24
4.4.2 Complete Source: `15_formatting.tcl` . . . 24
4.4.3 Expected Output . . . 24
4.4.4 Common Format Specifiers . . . 25
4.5 16 — String Parsing . . . 25
4.5.1 Concept . . . 25
4.5.2 Complete Source: `16_string_parsing.tcl` . . . 25
4.5.3 Expected Output . . . 25
4.5.4 Field Mapping . . . 25
4.6 17 — Basic Text Processing . . . 26
4.6.1 Concept . . . 26
4.6.2 Complete Source: `17_text_processing.tcl` . . . 26
4.6.3 Expected Output . . . 26
4.7 Command Reference . . . 26
4.8 Practical Applications . . . 27
4.9 Example Automation Flow . . . 27
4.10 Review Exercises . . . 27
4.11 Phase 04 Summary . . . 28

**5 File Automation 29**
5.1 Introduction . . . 29
5.2 Learning Objectives . . . 29
5.3 Folder Structure . . . 29
5.4 18 — Creating a File . . . 30
5.4.1 Concept . . . 30
5.4.2 Complete Source: `18_create_file.tcl` . . . 30
5.4.3 Expected Console Output . . . 30
5.4.4 Resulting File . . . 30
5.5 19 — Reading a File . . . 31
5.5.1 Concept . . . 31
5.5.2 Input File: `input.txt` . . . 31
5.5.3 Complete Source: `19_read_file.tcl` . . . 31
5.5.4 Expected Console Output . . . 31
5.5.5 Understanding Relative Paths . . . 31

5.5.6 Important File-Handling Rule . . . 32
5.6 20 — Writing Multiple Lines to a File . . . 32
5.6.1 Concept . . . 32
5.6.2 Complete Source: `20_write_file.tcl` . . . 32
5.6.3 Expected Console Output . . . 32
5.6.4 Generated Report Contents . . . 32
5.6.5 Why This Is Useful . . . 33
5.7 21 — Directory Operations . . . 33
5.7.1 Concept . . . 33
5.7.2 Complete Source: `21_directory_operations.tcl` . . . 33
5.7.3 Expected Output on the First Run . . . 33
5.7.4 Expected Output on Subsequent Runs . . . 34
5.7.5 Explanation . . . 34
5.7.6 A Note About `glob` . . . 34
5.8 22 — Generating a File Report . . . 34
5.8.1 Concept . . . 34
5.8.2 Complete Source: `22_file_report.tcl` . . . 34
5.8.3 Expected Console Output . . . 35
5.8.4 Generated File Contents . . . 35
5.8.5 Execution Flow . . . 35
5.9 File Modes and Important Commands . . . 35
5.9.1 File Modes . . . 35
5.9.2 Command Reference . . . 36
5.10 Practical Automation and EDA Applications . . . 36
5.11 Common Issues and Good Practices . . . 36
5.11.1 File Not Found . . . 36
5.11.2 Always Close File Channels . . . 37
5.11.3 Understand Write Mode . . . 37
5.11.4 Expect Directory Output to Change . . . 37
5.12 Review Exercises . . . 37
5.12.1 Repository Exercises . . . 37
5.12.2 Review Questions . . . 37
5.13 Chapter Summary . . . 38

**6 Pattern Matching 39**
6.1 Introduction . . . 39
6.2 Learning Objectives . . . 39
6.3 23 — Glob Matching . . . 39
6.3.1 Concept . . . 39
6.3.2 The `-nocomplain` Option . . . 40
6.3.3 Complete Source: `23_glob_matching.tcl` . . . 40
6.3.4 Expected Output . . . 40
6.3.5 Working Directory Note . . . 40
6.4 24 — Regular Expressions . . . 40
6.4.1 Concept . . . 40
6.4.2 Basic Pattern Elements . . . 40

6.4.3 Complete Source: `24_regular_expressions.tcl` . . . 41
6.4.4 Expected Output . . . 41
6.4.5 Capturing Groups and the `->` Variable . . . 41
6.5 25 — Log Parser . . . 42
6.5.1 Concept . . . 42
6.5.2 Input File: `test.log` . . . 42
6.5.3 Complete Source: `25_log_parser.tcl` . . . 42
6.5.4 Expected Output . . . 42
6.5.5 How the Loop Works . . . 42
6.5.6 Path and Execution Note . . . 43
6.6 26 — Data Extractor . . . 43
6.6.1 Concept . . . 43
6.6.2 Complete Source: `26_data_extractor.tcl` . . . 43
6.6.3 Expected Output . . . 43
6.6.4 Understanding the Pattern . . . 43
6.6.5 A Practical Limitation . . . 44
6.7 Glob and Regular Expressions Compared . . . 44
6.8 Command Reference . . . 44
6.9 Pattern-Matching Workflow . . . 44
6.10 Practical EDA Automation Uses . . . 44
6.11 Review Exercises . . . 45
6.12 Phase 06 Summary . . . 45

**7 Error Handling 46**
7.1 Introduction . . . 46
7.2 Learning Objectives . . . 46
7.3 The `catch` Command . . . 46
7.3.1 Purpose . . . 46
7.3.2 Syntax . . . 46
7.4 27 — Basic Error Handling with `catch` . . . 47
7.4.1 Objective . . . 47
7.4.2 Complete Source: `27_catch.tcl` . . . 47
7.4.3 Expected Output . . . 47
7.4.4 Step-by-Step Explanation . . . 47
7.5 28 — Handling a File-Operation Error . . . 48
7.5.1 Objective . . . 48
7.5.2 Complete Source: `28_error_handling.tcl` . . . 48
7.5.3 Expected Output . . . 48
7.5.4 Understanding the Conditional . . . 48
7.5.5 Why the File Is Closed Only on Success . . . 49
7.5.6 Relative-Path Note . . . 49
7.6 29 — Input Validation . . . 49
7.6.1 Objective . . . 49
7.6.2 Complete Source: `29_validation.tcl` . . . 49
7.6.3 Expected Output . . . 49
7.6.4 Checking for an Empty String . . . 50

7.6.5 Checking an Allowed Status List . . . 50
7.7 String, List, and Numeric Validation . . . 50
7.7.1 String Comparison Operators . . . 50
7.7.2 List-Based Validation . . . 50
7.7.3 Numeric Comparison Operators . . . 50
7.8 Error Handling Versus Input Validation . . . 51
7.9 A General Error-Handling Workflow . . . 52
7.10 Important Commands and Operators . . . 52
7.11 Practical EDA Automation Applications . . . 52
7.12 Common Mistakes and Checks . . . 53
7.13 Review Exercises . . . 53
7.14 Phase 07 Summary . . . 54

**8 Automation Utilities 55**
8.1 Introduction . . . 55
8.2 Learning Objectives . . . 55
8.3 30 — Executing Operating-System Commands . . . 55
8.3.1 Concept . . . 55
8.3.2 Complete Source: `30_command_execution.tcl` . . . 55
8.3.3 Expected Output . . . 56
8.3.4 Step-by-Step Explanation . . . 56
8.3.5 Important Consideration . . . 56
8.4 31 — Environment Variables . . . 56
8.4.1 Concept . . . 56
8.4.2 Complete Source: `31_environment_variables.tcl` . . . 57
8.4.3 Expected Output . . . 57
8.4.4 Understanding `$::env(NAME)` . . . 57
8.4.5 Environment Variables in EDA . . . 57
8.5 32 — Command-Line Arguments . . . 57
8.5.1 Concept . . . 57
8.5.2 Complete Source: `32_arguments.tcl` . . . 58
8.5.3 Running the Script . . . 58
8.5.4 Expected Output with Two Arguments . . . 58
8.5.5 Expected Output with No Arguments . . . 58
8.5.6 Accessing Individual Arguments . . . 59
8.6 33 — Complete Automation Script . . . 59
8.6.1 Concept . . . 59
8.6.2 Complete Source: `33_automation_script.tcl` . . . 59
8.6.3 How the Script Works . . . 60
8.6.4 Running the Script . . . 60
8.6.5 Expected Console Output . . . 60
8.6.6 Generated File: `report.txt` . . . 60
8.6.7 Argument Validation Examples . . . 61
8.6.8 Relative-Path Consideration . . . 61
8.7 The `exit` Command and Exit Status . . . 61
8.7.1 Purpose . . . 61

8.8 Automation Workflow . . . 61
8.9 Command and Variable Reference . . . 62
8.10 Practical EDA Automation Applications . . . 62
8.11 Common Mistakes and Checks . . . 62
8.12 Review Exercises . . . 63
8.13 Phase 08 Summary . . . 63

**9 EDA Workflows 65**
9.1 Introduction . . . 65
9.2 Learning Objectives . . . 65
9.3 34 — EDA File Reporter . . . 65
9.3.1 Concept . . . 65
9.3.2 Complete Source: `34_file_reporter.tcl` . . . 66
9.3.3 Expected Output . . . 66
9.3.4 Step-by-Step Explanation . . . 66
9.3.5 Important Commands . . . 66
9.3.6 EDA Applications . . . 67
9.4 35 — EDA Log Analyzer . . . 67
9.4.1 Concept . . . 67
9.4.2 Input File: `sample.log` . . . 67
9.4.3 Complete Source: `35_log_analyzer.tcl` . . . 67
9.4.4 Expected Output . . . 68
9.4.5 How the Parser Works . . . 68
9.4.6 EDA Applications . . . 68
9.5 36 — Timing Report Parser . . . 69
9.5.1 Concept . . . 69
9.5.2 Input File: `timing.rpt` . . . 69
9.5.3 Complete Source: `36_timing_report_parser.tcl` . . . 69
9.5.4 Expected Output . . . 70
9.5.5 Regular Expression Breakdown . . . 70
9.5.6 Timing Metrics . . . 70
9.5.7 Status Rule Used by the Exercise . . . 70
9.6 37 — Cell Statistics . . . 71
9.6.1 Concept . . . 71
9.6.2 Complete Source: `37_cell_statistics.tcl` . . . 71
9.6.3 Example Input Format . . . 71
9.6.4 Illustrative Output for the Example Input . . . 72
9.6.5 Important Note About the Supplied Archive . . . 72
9.6.6 Step-by-Step Explanation . . . 72
9.6.7 Assumptions and Limitations . . . 72
9.7 38 — Complete EDA Automation Flow . . . 72
9.7.1 Concept . . . 72
9.7.2 Complete Source: `38_eda_automation_flow.tcl` . . . 73
9.7.3 Running the Script . . . 74
9.7.4 Expected Console Output . . . 74
9.7.5 Generated Summary: `UART_summary.rpt` . . . 74

9.7.6 Execution Stages . . . 74
9.7.7 Path Consideration . . . 75
9.7.8 Robustness Considerations . . . 75
9.8 Workflow Architecture . . . 75
9.9 Command Reference . . . 75
9.10 Practical EDA Applications . . . 76
9.11 Common Mistakes and Checks . . . 76
9.12 Review Exercises . . . 76
9.13 Phase 09 Summary . . . 77

**10 Practical HDL Automation with Tcl 78**
10.1 Introduction . . . 78
10.2 Learning Objectives . . . 78
10.3 Toolchain and Automation Architecture . . . 78
10.3.1 Tools Used . . . 78
10.3.2 End-to-End Flow . . . 79
10.4 Project 1 — D Flip-Flop . . . 79
10.4.1 Project Files . . . 79
10.4.2 RTL: `D_flip_flop_behav.v` . . . 79
10.4.3 RTL Explanation . . . 80
10.4.4 Testbench: `D_flip_flop_tb.v` . . . 80
10.4.5 Tcl Runner: `run.tcl` . . . 80
10.4.6 How the Tcl Runner Works . . . 81
10.4.7 Recorded Output . . . 81
10.4.8 Run and Inspect . . . 82
10.5 Project 2 — 4-Bit ALU . . . 82
10.5.1 Project Files . . . 82
10.5.2 RTL: `ALU_4_bit.v` . . . 82
10.5.3 Operation Map . . . 82
10.5.4 Testbench: `ALU_4_bit_tb.v` . . . 83
10.5.5 Tcl Runner: `02_alu_4_bit/run.tcl` . . . 84
10.5.6 Recorded Simulation Results . . . 85
10.5.7 Report Generation . . . 85
10.5.8 Run and Inspect . . . 85
10.6 Project 3 — UART Transmitter and Receiver . . . 86
10.6.1 Project Files . . . 86
10.6.2 Transmitter RTL: `UART_Tx.v` . . . 86
10.6.3 Receiver RTL: `UART_Rx.v` . . . 87
10.6.4 Testbench: `UART_tb.v` . . . 87
10.6.5 UART Tcl Runner: `run.tcl` . . . 88
10.6.6 Recorded Output and Important Functional Note . . . 89
10.6.7 Report Generation . . . 89
10.6.8 Run and Inspect . . . 89
10.7 Running the Projects in Ubuntu/WSL . . . 90
10.7.1 Navigate to the Project Directory . . . 90
10.8 Understanding `exec` and `catch` . . . 90

10.8.1 `exec` . . . 90
10.8.2 `catch` . . . 90
10.8.3 Important Limitation . . . 90
10.9 Logs, Reports, and Waveforms . . . 91
10.10 Troubleshooting . . . 91
10.11 Practical Improvements . . . 91
10.12 Command Reference . . . 92
10.13 Review Exercises . . . 92
10.14 Phase 10 Summary . . . 93

# Chapter 1

# Core Tcl Syntax

## 1.1 Introduction

This chapter introduces the core Tcl syntax used throughout the Tcl Scripting and EDA Journey. The Phase 01 repository exercises cover variables and output, arithmetic expressions, conditional statements, loops, and basic list operations. These fundamentals provide the building blocks for later data handling, file automation, and practical HDL workflows.

## 1.2 Learning Objectives

After completing this chapter, you should be able to:

- create variables and display their values with `set` and `puts`;
- evaluate arithmetic expressions with `expr`;
- use `if`, `elseif`, and `else` for decisions;
- repeat commands with `for` and `while` loops;
- use `incr` to update loop counters;
- retrieve and append elements in Tcl lists; and
- recognize variable substitution, command substitution, and grouping with braces.

## 1.3 Repository Exercises

The Phase 01 source archive contains these five Tcl scripts:

- `01_variables_and_output.tcl` — variables and basic output;
- `02_expressions.tcl` — arithmetic and expressions;
- `03_conditions.tcl` — conditional statements;
- `04_loops.tcl` — `for` and `while` loops; and
- `05_lists.tcl` — basic list operations.

The source listings and outputs in this chapter follow the repository scripts. Additional explanations clarify the syntax used in those examples.

## 1.4 Variables and Basic Output

Tcl variables are created or updated with `set`. The `puts` command writes text to the terminal. Inside double quotes, a variable reference such as `$name` is substituted with the variable's value.

### 1.4.1 Complete source: `01_variables_and_output.tcl`

```tcl
# Variables and basic output

set name "Bibhu"
set role "Tcl Learner"
set version 9.0

# Variables are accessed using the $ prefix inside the string.
puts "Name    : $name"
puts "Role    : $role"
puts "Tcl     : $version"
```

Listing 1.1: Variables and basic output

### 1.4.2 Expected output

```
Name    : Bibhu
Role    : Tcl Learner
Tcl     : 9.0
```

### 1.4.3 Command reference

| Command | Purpose |
|---|---|
| set | Creates a variable or assigns a new value to an existing variable. |
| puts | Writes a string or value to standard output by default. |

## 1.5 Expressions and Arithmetic

The `expr` command evaluates an expression. Square brackets perform command substitution: Tcl evaluates the command inside `[]` and substitutes its result into the surrounding command. The expression is enclosed in braces to keep it grouped and allow `expr` to perform the substitutions and evaluation safely.

### 1.5.1 Complete source: `02_expressions.tcl`

```tcl
# Basic arithmetic and expressions

set a 20
set b 6

# expr evaluates the arithmetic expression; [ ] performs command substitution.
set sum [expr {$a + $b}]
set difference [expr {$a - $b}]
set product [expr {$a * $b}]
set quotient [expr {$a / $b}]
set remainder [expr {$a % $b}]
```

```tcl

puts "A = $a"
puts "B = $b"
puts "Sum        : $sum"
puts "Difference : $difference"
puts "Product    : $product"
puts "Quotient   : $quotient"
puts "Remainder  : $remainder"
```

Listing 1.2: Basic arithmetic and expressions

### 1.5.2 Expected output

```
A = 20
B = 6
Sum        : 26
Difference : 14
Product    : 120
Quotient   : 3
Remainder  : 2
```

### 1.5.3 Arithmetic operators

| Operator | Operation |
|---|---|
| + | Addition. |
| - | Subtraction. |
| * | Multiplication. |
| / | Division. |
| % | Remainder after division. |

Because both operands in `$a / $b` are integers, the example produces integer division: `20 / 6` evaluates to `3`, not a decimal fraction.

## 1.6 Conditional Statements

Conditional statements select which commands to execute based on a condition. Tcl supports `if`, `elseif`, and `else`. In the repository example, the temperature value is tested against two thresholds.

### 1.6.1 Complete source: `03_conditions.tcl`

```tcl
# Conditional statements

set temperature 28

# Conditions are enclosed in braces and evaluated by Tcl.
if {$temperature >= 30} {
```

```
    puts "Temperature: Hot"
} elseif {$temperature >= 20} {
    puts "Temperature: Moderate"
} else {
    puts "Temperature: Cold"
}
```

Listing 1.3: Conditional statements

### 1.6.2 Expected output

```
Temperature: Moderate
```

### 1.6.3 Comparison operators

| Operator | Meaning |
|---|---|
| == | Equal to. |
| != | Not equal to. |
| > | Greater than. |
| < | Less than. |
| >= | Greater than or equal to. |
| <= | Less than or equal to. |

Tcl checks the `if` condition first. If it is false, the `elseif` condition is checked. The `else` block runs only when the preceding conditions are false.

## 1.7 Loops

Loops repeat a block of commands. The repository example demonstrates both a `for` loop and a `while` loop, each printing the values from 1 through 5.

### 1.7.1 Complete source: `04_loops.tcl`

```
# For and while loops

puts "For loop:"

# for uses initialization, condition, and update in one statement.
for {set i 1} {$i <= 5} {incr i} {
    puts "Count: $i"
}

puts ""
puts "While loop:"

set j 1

```

```tcl
# while repeats the block as long as the condition remains true.
while {$j <= 5} {
    puts "Count: $j"

    # incr increases the loop counter by 1.
    incr j
}
```

Listing 1.4: For and while loops

### 1.7.2 Expected output

```
For loop:
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5

While loop:
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5
```

### 1.7.3 Loop structure

A `for` loop has three parts: initialization, condition, and update. In this example, `set i 1` initializes the counter, `$i <= 5` is the condition, and `incr i` updates the counter after each iteration.

A `while` loop checks its condition before each iteration. The `incr j` command is inside the loop body, so the counter advances and the condition eventually becomes false. If a loop's condition never becomes false, the loop may run indefinitely.

### 1.7.4 Loop command reference

| Command | Purpose |
|---|---|
| for | Repeats a command body using initialization, condition, and update scripts. |
| while | Repeats a command body while a condition evaluates as true. |
| incr | Increments an integer variable; an optional second argument specifies the increment amount. |

## 1.8 Basic Tcl Lists

A Tcl list is an ordered collection of elements. The repository example stores tool names in a list, checks its length, retrieves the first and last elements, and appends another item.

### 1.8.1 Complete source: `05_lists.tcl`

```
# Basic Tcl list operations

set tools {Tcl Verilog Python Git}

puts "Tools: $tools"

# llength returns the number of elements in a list.
puts "Number of tools: [llength $tools]"

# lindex accesses a list element using its index; Tcl indexes start from 0.
puts "First tool: [lindex $tools 0]"

# "end" refers to the last element of the list.
puts "Last tool: [lindex $tools end]"

# lappend adds a new element to an existing list.
lappend tools Linux

puts "After adding Linux:"
puts $tools
```

Listing 1.5: Basic Tcl list operations

### 1.8.2 Expected output

```
Tools: Tcl Verilog Python Git
Number of tools: 4
First tool: Tcl
Last tool: Git
After adding Linux:
Tcl Verilog Python Git Linux
```

### 1.8.3 List command reference

| Command | Purpose |
|---|---|
| llength | Returns the number of elements in a list. |
| lindex | Retrieves the element at a specified index. |
| lappend | Appends one or more elements to a list variable. |

List indexes start at zero. Therefore, `[lindex $tools 0]` returns the first element, while `[lindex $tools end]` returns the last element. The `lappend` command updates the list variable by adding `Linux` as a new element.

## 1.9 Tcl Substitution and Grouping

Tcl uses a few important syntax forms when parsing commands:

- **Variable substitution:** `$name` is replaced with the value of the variable `name`.
- **Command substitution:** `[command]` executes the command and substitutes its result.
- **Braces:** `{}` group words or script blocks and generally prevent variable and command substitutions during the initial parsing of their contents.
- **Double quotes:** `" "` group text into one argument while allowing variable and command substitutions.

### 1.9.1 Examples

```
set name "Bibhu"
puts "Hello $name"
puts [expr {10 + 5}]
puts {$name}
```

Listing 1.6: Variable, command, and brace substitution

### 1.9.2 Expected output

```
Hello Bibhu
15
$name
```

The first command displays the substituted variable value. The second executes `expr` inside square brackets and displays its result. The third uses braces, so the text `$name` is printed literally.

## 1.10 Key Concepts to Remember

1. Tcl commands consist of a command name followed by arguments.
2. Use `set` to create or update a variable and `puts` to display output.
3. Use `expr` to evaluate arithmetic expressions.
4. Square brackets perform command substitution.
5. Use `if`, `elseif`, and `else` to make decisions.
6. Use `for` and `while` to repeat commands.
7. Use `incr` to update integer counters.
8. Tcl list indexes begin at zero.
9. Use `llength`, `lindex`, and `lappend` for the list operations covered in this phase.
10. Double quotes permit substitutions; braces are useful for grouping expressions and script blocks.

## 1.11 Review Exercises

Use the repository files as a starting point for these exercises:

1. In `01_variables_and_output.tcl`, add another variable and display it with a descriptive label.
2. In `02_expressions.tcl`, change the values of `a` and `b`; predict the integer quotient and remainder before running the script.
3. In `03_conditions.tcl`, test values below 20, from 20 through 29, and 30 or above.
4. In `04_loops.tcl`, modify both loops to count from 1 through 10.
5. In `05_lists.tcl`, append another tool and print the updated list length.

## 1.12 Phase 01 Summary

Phase 01 establishes the Tcl foundations used by the remaining phases: command structure, variables, output, expressions, conditions, loops, and basic lists. The repository's five scripts provide small runnable examples that can be modified and tested individually.
**Phase 01 complete.** Next: Phase 02 — Data Handling.

# Chapter 2

# Data Handling

## 2.1 Introduction

This phase explores how Tcl stores, retrieves, searches, and modifies collections and structured information. The repository exercises cover advanced list operations, arrays, dictionaries, and nested data. These structures are useful in automation scripts that manage module names, test statuses, configuration values, and project reports.

## 2.2 Learning Objectives

After completing this chapter, you should be able to:

- use Tcl list commands to inspect and modify ordered collections;
- store and retrieve values using array keys;
- create and update dictionaries;
- combine a dictionary with a list to represent nested data; and
- select an appropriate structure for a small automation task.

## 2.3 Repository Exercises

The source archive contains these four Tcl scripts:

- `06_list_operations.tcl` — list manipulation;
- `07_arrays.tcl` — key-based array storage;
- `08_dictionaries.tcl` — dictionary operations; and
- `09_nested_data.tcl` — a dictionary containing a list.

The listings and expected outputs below follow those source scripts. For arrays, the order returned by `array names` is not guaranteed, so the displayed key order can vary.

## 2.4 Lists and Advanced List Operations

A Tcl list is an ordered collection of elements. List indexes begin at zero. The first script uses module names to demonstrate common list operations.

### 2.4.1 Complete source: `06_list_operations.tcl`

```tcl
# Advanced Tcl list operations

set modules {ALU FIFO UART SPI I2C}

# llength returns the number of elements in the list.
puts "Number of modules: [llength $modules]"

# lindex retrieves an element using its index.
puts "Module at index 2: [lindex $modules 2]"

# lrange extracts a range of elements from a list.
puts "First three modules: [lrange $modules 0 2]"

# lsearch returns the index of a matching element.
puts "Index of SPI: [lsearch $modules SPI]"

# linsert inserts an element at a specified index.
set modules [linsert $modules 2 GPIO]
puts "After insertion: $modules"

# lreplace replaces one or more elements.
set modules [lreplace $modules 0 0 CPU]
puts "After replacement: $modules"

# lsort returns a sorted version of the list.
puts "Sorted modules: [lsort $modules]"
```

Listing 2.1: Advanced Tcl list operations

### 2.4.2 Command reference

| Command | Purpose |
|---|---|
| llength | Returns the number of elements in a list. |
| lindex | Retrieves an element at a specified index. |
| lrange | Extracts a range of elements. |
| lsearch | Searches for an element and returns its index; a missing element normally produces `-1`. |
| linsert | Inserts one or more elements at a specified position. |
| lreplace | Replaces or removes elements in a specified range. |
| lsort | Returns a sorted version of a list. |

### 2.4.3 Expected output

```
Number of modules: 5
Module at index 2: UART
First three modules: ALU FIFO UART
Index of SPI: 3
After insertion: ALU FIFO GPIO UART SPI I2C
```

```
After replacement: CPU FIFO GPIO UART SPI I2C
Sorted modules: CPU FIFO GPIO I2C SPI UART
```

### 2.4.4 How the list changes

The initial list is `{ALU FIFO UART SPI I2C}`. The `linsert` call inserts `GPIO` at index 2, before the original `UART`. The `lreplace` call then replaces the element at index 0, `ALU`, with `CPU`. Finally, `lsort` returns the alphabetically sorted values without changing the stored list.

## 2.5 Tcl Arrays

A Tcl array stores values under named keys. Array elements are accessed using the form `arrayName(key)`. The source example tracks pass/fail status for hardware modules.

### 2.5.1 Complete source: `07_arrays.tcl`

```
# Tcl arrays store data using named keys.

array set module_status {
    ALU  PASS
    FIFO PASS
    UART FAIL
    SPI  PASS
}

# Access an array element using its key.
puts "UART status: $module_status(UART)"

# array names returns all keys in the array.
puts "Modules: [array names module_status]"

# Check whether a particular key exists.
puts "SPI exists: [info exists module_status(SPI)]"

# Add or update an array element.
set module_status(I2C) PASS

puts "I2C status: $module_status(I2C)"
```

Listing 2.2: Array status tracking

### 2.5.2 Key commands

- `array set module_status {...}` initializes multiple key-value elements.
- `$module_status(UART)` reads the value associated with the `UART` key.
- `array names module_status` returns the keys currently stored in the array.
- `info exists module_status(SPI)` returns `1` if that element exists and `0` otherwise.
- `set module_status(I2C) PASS` adds the `I2C` element, or updates it if it already exists.

### 2.5.3 Expected output

```
UART status: FAIL
Modules: SPI UART FIFO ALU
SPI exists: 1
I2C status: PASS
```

**Note:** The order of names returned by `array names` is not guaranteed. The module-key line may appear in a different order on another run or Tcl implementation; the values and existence check are what matter.

## 2.6 Dictionaries

A Tcl dictionary stores key-value pairs together in one variable. It is suitable for representing related fields, such as a module's name, protocol, width, and status.

### 2.6.1 Complete source: `08_dictionaries.tcl`

```tcl
# Dictionaries store key-value pairs in a single variable.

set module {
    name UART
    protocol Serial
    width 8
    status PASS
}

# dict get retrieves a value using its key.
puts "Module name: [dict get $module name]"
puts "Protocol: [dict get $module protocol]"
puts "Width: [dict get $module width]"

# dict set adds or updates a key-value pair.
dict set module frequency 115200

puts "Frequency: [dict get $module frequency]"

# dict keys returns all keys in the dictionary.
puts "Keys: [dict keys $module]"
```

Listing 2.3: Creating and updating a dictionary

### 2.6.2 Key commands

- `dict get $module name` retrieves the value associated with `name`.
- `dict set module frequency 115200` adds the `frequency` key with value `115200`.
- `dict keys $module` returns the dictionary's keys.

### 2.6.3 Expected output

```
Module name: UART
Protocol: Serial
Width: 8
Frequency: 115200
Keys: name protocol width status frequency
```

## 2.7 Nested Data

Tcl structures can be combined. In this exercise, one dictionary stores project metadata, and one of its values is a list of protocols. The list can be retrieved and then processed using ordinary list commands.

### 2.7.1 Complete source: `09_nested_data.tcl`

```tcl
# Nested data can represent structured information.

set project [dict create \
    name "UART Controller" \
    language "Verilog" \
    protocols {UART SPI I2C} \
    status "Completed"
]

# Retrieve simple dictionary values.
puts "Project: [dict get $project name]"
puts "Language: [dict get $project language]"

# Retrieve a list stored inside the dictionary.
set protocols [dict get $project protocols]

puts "Protocols: $protocols"
puts "Number of protocols: [llength $protocols]"

# Access an individual item from the nested list.
puts "First protocol: [lindex $protocols 0]"

puts "Status: [dict get $project status]"
```

Listing 2.4: A dictionary containing a list

### 2.7.2 Expected output

```
Project: UART Controller
Language: Verilog
Protocols: UART SPI I2C
Number of protocols: 3
First protocol: UART
Status: Completed
```

### 2.7.3 Understanding the nested structure

The dictionary has four keys: `name`, `language`, `protocols`, and `status`. The value stored at `protocols` is itself the Tcl list `{UART SPI I2C}`. The script first extracts this value with `dict get`, then uses `llength` to count the protocols and `lindex` to retrieve the first protocol.

## 2.8 Choosing Between Lists, Arrays, and Dictionaries

| Structure | Purpose | Example |
|---|---|---|
| List | Ordered collection of values. | `set tools {Tcl Verilog Python Git}` |
| Array | Key-based collection whose elements are accessed like separate variables. | `set status(UART) PASS` |
| Dictionary | Key-value data stored together in one variable. | `set module {name UART status PASS}` |

A practical rule of thumb is:

- **List:** use when order and position matter.
- **Array:** use when looking up a value by a key is convenient and array-style elements are appropriate.
- **Dictionary:** use when related key-value fields should be grouped in one value that can be passed around and updated.
- **Nested data:** combine structures when a field itself contains a collection or more detailed record.

## 2.9 Practical Automation Uses

### 2.9.1 Lists

Lists can hold file names, module names, signal names, tool names, source files, or processing queues.

### 2.9.2 Arrays

Arrays can support status tracking, key-based lookup, counters, configuration values, and per-module information.

### 2.9.3 Dictionaries

Dictionaries can represent configuration data, structured reports, module information, tool settings, or parsed data.

### 2.9.4 Nested structures

Nested structures are useful when information has multiple levels, such as a project containing module records, each with a name, status, protocols, and configuration.

## 2.10 Review Exercises

1. Create a list of five module names. Print its length, the element at index 2, and the final element.
2. Search a list for an item that is not present. What value does `lsearch` normally return?
3. Extend the list exercise by inserting one module and replacing another.
4. Create an array containing at least three module statuses. Check whether one key exists, then add a new module status.
5. Create a dictionary for a module with keys `name`, `protocol`, `width`, and `status`. Print each value.
6. Add a `frequency` key to the dictionary and print all dictionary keys.
7. Create a project dictionary whose `protocols` value is a list. Retrieve the list, count its elements, and print the first element.

## 2.11 Important Commands at a Glance

| Command | Purpose |
|---|---|
| llength | Count list elements. |
| lindex | Retrieve a list element by index. |
| lrange | Extract a range from a list. |
| lsearch | Find an element's index. |
| linsert | Insert list elements. |
| lreplace | Replace a range of list elements. |
| lsort | Return a sorted list. |
| array set | Initialize multiple array elements. |
| array names | Return array keys. |
| info exists | Test whether a variable or array element exists. |
| dict create | Create a dictionary from key-value pairs. |
| dict get | Retrieve a dictionary value. |
| dict set | Add or update a dictionary entry. |
| dict keys | Return dictionary keys. |

## 2.12 Chapter Summary

This phase covered list manipulation, arrays, dictionaries, and nested data. Lists represent ordered values; arrays provide key-based access through array elements; dictionaries group

key-value pairs in a single variable; and nested structures combine these forms to represent more complex information. These concepts provide a foundation for later Tcl scripts that process files, HDL modules, configuration data, and simulation reports.

**Phase 02 complete.** The next phase is **Phase 03 — Procedures**, which focuses on reusable procedures, arguments, return values, and variable scope.

# Chapter 3

# Procedures

## 3.1 Introduction

Tcl procedures package commands into named, reusable operations. They help reduce duplicated code and make scripts easier to read, test, and maintain. In automation workflows, procedures can be reused for tasks such as processing files, checking results, parsing reports, and generating summaries.

This chapter follows the Phase 03 repository files. The examples and expected outputs below are based on the supplied scripts and README. Where the README provides an additional teaching example, it is identified as an explanatory example rather than a separate repository file.

## 3.2 Learning Objectives

By the end of this chapter, you should be able to:

- Define and call a Tcl procedure using `proc`.
- Declare procedure parameters and pass arguments.
- Return a value with `return` and use that value in a caller.
- Explain the difference between local and global variable scope.
- Use `global` when a procedure needs to access a global variable.
- Recognize how procedures support maintainable automation scripts.

## 3.3 Repository Files

The uploaded `03_procedures` folder contains these learning scripts:

| File | Main concept |
|---|---|
| `10_basic_procedure.tcl` | Defining and calling a procedure with no parameters |
| `11_arguments.tcl` | Passing arguments into a procedure |
| `12_return_values.tcl` | Returning and storing a calculated value |
| `13_scope.tcl` | Local variable scope inside a procedure |
| `README.md` | Phase overview, global variables, reference examples, and exercises |

## 3.4 Procedure Basics

### 3.4.1 What is a procedure?

A procedure is a named block of Tcl commands. The `proc` command defines it. Its general form is:

```
proc procedure_name {parameters} {
    # Procedure body
}
```

The parameter list is empty when no input values are required. Defining a procedure does not run its body; the procedure runs when its name is called.

### 3.4.2 Repository example: basic procedure

**Source file:** `10_basic_procedure.tcl`

```
# Basic Tcl procedure

# proc creates a reusable procedure with a name, parameters, and body.
proc greet {} {
    puts "Hello from Tcl procedure!"
}

# Call the procedure by using its name.
greet
```

Listing 3.1: Complete repository source: 10_basic_procedure.tcl

**Expected output:**

```
Hello from Tcl procedure!
```

**Explanation:**

- `proc greet {}` defines a procedure named `greet` with no parameters.
- The `puts` command is executed when the procedure is called.
- The final `greet` command invokes the procedure.

## 3.5 Procedure Arguments

### 3.5.1 Parameters and arguments

Parameters are the names declared in a procedure definition. Arguments are the values supplied when the procedure is called. The values are assigned to the corresponding parameters for that call.

### 3.5.2 Repository example: passing arguments

**Source file:** `11_arguments.tcl`

```tcl
# Procedures with arguments

# Arguments allow a procedure to receive values from the caller.
proc greet {name role} {
    puts "Name: $name"
    puts "Role: $role"
}

greet "Bibhu" "Tcl Learner"
```

Listing 3.2: Complete repository source: 11_arguments.tcl

**Expected output:**

```
Name: Bibhu
Role: Tcl Learner
```

Here, `name` and `role` are parameters. The strings `Bibhu` and `Tcl Learner` are the arguments. Inside the procedure, `$name` and `$role` substitute the values received from the caller.

## 3.6 Returning Values

### 3.6.1 The `return` command

A procedure can produce a value for the calling code. The `return` command ends the procedure and passes its result back to the caller. That result can be stored in a variable or used directly in another command.

### 3.6.2 Repository example: return a calculation

**Source file:** `12_return_values.tcl`

```tcl
# Procedures with return values

# return sends a value back to the caller.
proc add_numbers {a b} {
    return [expr {$a + $b}]
}

# Store the value returned by the procedure.
set result [add_numbers 20 15]

puts "Result: $result"
```

Listing 3.3: Complete repository source: 12_return_values.tcl

**Expected output:**

```
Result: 35
```

The call `add_numbers 20 15` passes two values to the procedure. The expression adds them, `return` sends the result back, and `set result` stores that value.

### 3.6.3 Additional example from the phase README

The README also shows that a returned value can be used directly inside another command:

```
puts "Result: [add_numbers 5 10]"
```

**Output:**

```
Result: 15
```

The square brackets evaluate the procedure call first; the resulting value is then inserted into the `puts` command.

## 3.7 Variable Scope

### 3.7.1 Local variables

Variables created inside a procedure are local by default. A local variable exists in that procedure's scope and does not overwrite a global variable with the same name.

### 3.7.2 Repository example: local and global names

**Source file:** `13_scope.tcl`

```
# Variable scope in procedures

set message "Global message"

proc show_message {} {
    # A variable created inside a procedure is local to that procedure.
    set message "Local message"
    puts "Inside procedure: $message"
}

show_message

puts "Outside procedure: $message"
```

Listing 3.4: Complete repository source: 13_scope.tcl

**Expected output:**

```
Inside procedure: Local message
Outside procedure: Global message
```

The procedure creates and prints a local variable named `message`. The global `message` remains unchanged, so the final `puts` prints `Global message`.

### 3.7.3 Accessing a global variable

The phase README introduces the `global` command, which links a variable name inside a procedure to a global variable.

```tcl
set counter 0

proc increment_counter {} {
    global counter
    incr counter
}

increment_counter
puts $counter
```

**Output:**

```
1
```

The `global counter` declaration lets `incr counter` update the global variable. This is an additional README example; it is not one of the four numbered `.tcl` files in the uploaded folder.

## 3.8 Procedure Structure and Reuse

A common procedure structure is:

```tcl
proc procedure_name {arguments} {
    # Perform the required processing.
    return result
}
```

For example, the README demonstrates multiplication:

```tcl
proc multiply {a b} {
    return [expr {$a * $b}]
}

set result [multiply 5 4]
puts $result
```

**Output:**

```
20
```

This is a README teaching example, not an additional repository script. The same pattern can be adapted to other repeated calculations or automation tasks.

## 3.9 Important Commands Reference

| Command | Purpose in this chapter |
|---|---|
| `proc` | Defines a named procedure with parameters and a body. |
| `return` | Returns a value from a procedure and ends its execution. |
| `global` | Makes a global variable accessible by name inside a procedure. |

| | |
|---|---|
| `expr` | Evaluates an expression, such as arithmetic on procedure arguments. |
| `incr` | Increments a numeric variable; the README uses it in the global-counter example. |
| `set` | Creates or assigns a variable. |
| `puts` | Writes a message or value to standard output. |

## 3.10 Practical Applications in Automation

Procedures are useful when the same operation must be repeated with different inputs. In larger Tcl scripts, they can organize operations such as:

- processing multiple files;
- parsing report data;
- checking module or run status;
- calculating statistics;
- generating reports;
- validating input data; and
- grouping repeated tool commands into reusable steps.

These are general applications described by the phase README; the four supplied scripts focus on procedure definition, arguments, return values, and scope.

## 3.11 Review Exercises

1. Run `10_basic_procedure.tcl`. Identify the definition and the call.
2. In `11_arguments.tcl`, identify the parameters and the arguments.
3. Run `12_return_values.tcl`. Explain how the returned value reaches `result`.
4. Run `13_scope.tcl`. Why does the global message remain unchanged?
5. Modify the README's counter example to call `increment_counter` three times. Predict the final value, then test it.
6. Write a procedure that accepts two numbers and returns their product. Store the result and print it.

## 3.12 Phase Summary

In Phase 03, you learned to create and call Tcl procedures, pass parameters, return values, and understand local versus global scope. These concepts help transform a long script into smaller, reusable operations that are easier to maintain and extend.

**Phase 03 complete!**

**Next: Phase 04 — String Processing.**

# Chapter 4

# String Processing

## 4.1 Introduction

String processing is used to inspect, transform, search, and extract information from text. Tcl automation frequently handles filenames, tool output, logs, configuration data, and reports. This phase covers basic string operations, formatting, parsing, and text manipulation.

Repository source files:

- `14_string_operations.tcl`
- `15_formatting.tcl`
- `16_string_parsing.tcl`
- `17_text_processing.tcl`

## 4.2 Learning Objectives

After completing this chapter, you should be able to:

- measure strings, convert case, access characters, and extract ranges;
- format values into readable messages;
- split delimited text into a Tcl list and retrieve fields;
- search for substrings, match wildcard patterns, and replace text;
- apply string commands to log processing and EDA automation.

## 4.3 14 — Basic String Operations

### 4.3.1 Concept

The `string` command provides subcommands for manipulating text. Tcl string indexes start at zero. `string length` returns the number of characters; `toupper` and `tolower` convert letter case; `index` returns one character; and `range` extracts characters between two indexes.

### 4.3.2 Complete Source: `14_string_operations.tcl`

```
# Basic Tcl string operations

set text "Tcl Automation"

# string length returns the number of characters.
puts "Length: [string length $text]"
```

```tcl

# string toupper converts text to uppercase.
puts "Uppercase: [string toupper $text]"

# string tolower converts text to lowercase.
puts "Lowercase: [string tolower $text]"

# string index accesses a character using its index.
puts "First character: [string index $text 0]"

# string range extracts characters between two indexes.
puts "First three characters: [string range $text 0 2]"
```

### 4.3.3 Expected Output

```
Length: 14
Uppercase: TCL AUTOMATION
Lowercase: tcl automation
First character: T
First three characters: Tcl
```

**Source consistency note:** the source script reports `Length: 14`, which is correct for `Tcl Automation`. The README's separate example says `15`; that README example is inconsistent with the actual string.

## 4.4 15 — String Formatting

### 4.4.1 Concept

The `format` command constructs a string using placeholders. In this script, `%s` represents a string and `%d` represents an integer. The formatted result is stored in `message` and printed.

### 4.4.2 Complete Source: `15_formatting.tcl`

```tcl
# String formatting

set name "Bibhu"
set project "Tcl Automation"
set version 1

# format creates a formatted string using placeholders.
set message [format "Name: %s | Project: %s | Version: %d" $name $project $version
    ]

puts $message
```

### 4.4.3 Expected Output

```
Name: Bibhu | Project: Tcl Automation | Version: 1
```

### 4.4.4 Common Format Specifiers

| Specifier | Purpose |
|---|---|
| %s | String value |
| %d | Integer value |
| %f | Floating-point value |

## 4.5 16 — String Parsing

### 4.5.1 Concept

The `split` command divides a string into a Tcl list using a delimiter. The script splits `UART,115200,PASS` at commas. The fields are retrieved with `lindex`, whose indexes begin at zero.

### 4.5.2 Complete Source: `16_string_parsing.tcl`

```
# String parsing

set data "UART,115200,PASS"

# split divides a string into a list using the specified separator.
set fields [split $data ","]

puts "Protocol: [lindex $fields 0]"
puts "Baud Rate: [lindex $fields 1]"
puts "Status: [lindex $fields 2]"
```

### 4.5.3 Expected Output

```
Protocol: UART
Baud Rate: 115200
Status: PASS
```

### 4.5.4 Field Mapping

| List index | Value |
|---|---|
| 0 | UART |
| 1 | 115200 |
| 2 | PASS |

The script assumes the first field is the protocol, the second is the baud rate, and the third is the status. If the record format changes, the extraction logic may need to change.

## 4.6 17 — Basic Text Processing

### 4.6.1 Concept

The source combines three operations:

- `string first` returns the starting index of a substring, or `-1` when it is not found;
- `string match` checks a string against a wildcard pattern. The `*` wildcard matches any sequence of characters;
- `string map` replaces specified text according to a mapping list.

### 4.6.2 Complete Source: `17_text_processing.tcl`

```tcl
# Basic text processing

set text "ERROR: UART communication failed"

# string first finds the position of a substring.
set position [string first "UART" $text]

puts "UART position: $position"

# string match checks whether text matches a pattern.
if {[string match "ERROR:*" $text]} {
    puts "Error message detected"
}

# string map replaces specified text.
set updated [string map {"ERROR:" "WARNING:"} $text]

puts "Updated text: $updated"
```

### 4.6.3 Expected Output

```
UART position: 7
Error message detected
Updated text: WARNING: UART communication failed
```

## 4.7 Command Reference

| Command | Purpose |
|---|---|
| `string length` | Returns the number of characters |
| `string toupper` | Converts a string to uppercase |
| `string tolower` | Converts a string to lowercase |
| `string index` | Returns the character at an index |
| `string range` | Extracts characters between two indexes |

| Command | Purpose |
|---|---|
| `format` | Builds a string from format specifiers and values |
| `split` | Splits a string into a list using a delimiter |
| `lindex` | Retrieves a list element by index |
| `string first` | Finds the first position of a substring |
| `string match` | Checks a string against a wildcard pattern |
| `string map` | Replaces specified text in a string |

## 4.8 Practical Applications

String processing can be used to:

- parse simulation and tool log messages;
- extract fields from comma-separated status records;
- detect error prefixes in output;
- replace message labels when preparing summaries;
- create readable status reports;
- process filenames, module names, and configuration values.

A record such as `UART,115200,PASS` can be split into fields, checked, and formatted into a report. The same approach can be adapted to other delimited text when the delimiter and field meanings are known.

## 4.9 Example Automation Flow

**Raw Text**

↓

`split` / `string` operations

↓

**Extract Required Information**

↓

**Check or Process Information**

↓

**Generate Formatted Output**

| Field | Value |
|---|---|
| Protocol | UART |
| Baud Rate | 115200 |
| Status | PASS |

## 4.10 Review Exercises

1. In `14_string_operations.tcl`, change the text and predict the results of `string length`, `string index`, and `string range`.

2. In `15_formatting.tcl`, add another value to the message using an appropriate format specifier.
3. In `16_string_parsing.tcl`, change the record to `SPI,8000000,PASS` and inspect the extracted fields.
4. Search for a substring that is absent and observe the result of `string first`.
5. Change the wildcard pattern in `17_text_processing.tcl` and test whether the condition executes.
6. Use `string map` to replace a different label in the message.
7. Explain why the indexes passed to `string index` and `lindex` begin at zero.

## 4.11 Phase 04 Summary

Phase 04 introduced string length and case conversion, character indexing, range extraction, formatted messages, delimiter-based parsing, substring searching, wildcard matching, and text replacement. These techniques are useful for processing logs, reports, configuration values, and other text-based automation data.

### Phase Completion Checklist

☐ Run `14_string_operations.tcl` and compare the output.

☐ Run `15_formatting.tcl` and inspect the formatted message.

☐ Run `16_string_parsing.tcl` and identify each extracted field.

☐ Run `17_text_processing.tcl` and verify the search, match, and replacement results.

**Next phase:** Phase 05 — File Automation.

# Chapter 5

# File Automation

## 5.1 Introduction

File automation is an important Tcl skill because scripts often need to create files, read existing data, write reports, inspect directories, and process tool-generated output. This phase introduces the core file and directory commands through five practical exercises.

The chapter follows the supplied `05_file_automation` folder and its README. The repository contains these scripts:

- `18_create_file.tcl`
- `19_read_file.tcl`
- `20_write_file.tcl`
- `21_directory_operations.tcl`
- `22_file_report.tcl`

The folder also includes `input.txt`, which is used by the file-reading exercise.

## 5.2 Learning Objectives

After completing this chapter, the reader should be able to:

- create or overwrite a file using `open`;
- read the contents of an existing file;
- write multiple lines into a report;
- close file channels after use;
- check whether a path exists and create a directory;
- find directory entries with `glob`;
- combine lists, `foreach`, and file operations to generate a report;
- understand how the current working directory affects relative paths.

## 5.3 Folder Structure

```
05_file_automation/
|-- 18_create_file.tcl
|-- 19_read_file.tcl
|-- 20_write_file.tcl
|-- 21_directory_operations.tcl
```

```
|-- 22_file_report.tcl
|-- input.txt
`-- README.md
```

## 5.4 18 — Creating a File

### 5.4.1 Concept

The Tcl `open` command opens a file and returns a channel identifier. The mode `w` opens a file for writing, creating it if necessary and overwriting existing contents. The `puts` command writes text to the channel, and `close` releases the file resource.

### 5.4.2 Complete Source: `18_create_file.tcl`

```tcl
# Creating a file

set filename "output.txt"

# open with "w" creates a new file or overwrites an existing file.
set file [open $filename w]

puts $file "Tcl file automation"

# close releases the file resource.
close $file

puts "File created: $filename"

# Output:
# File created: output.txt
```

### 5.4.3 Expected Console Output

```
File created: output.txt
```

### 5.4.4 Resulting File

The script creates `output.txt` in the current working directory and writes the following line into it:

```
Tcl file automation
```

The script prints a confirmation message after closing the file.

## 5.5 19 — Reading a File

### 5.5.1 Concept

The `read` command retrieves the contents of an opened file. The file must exist at the path supplied to `open`. In the repository, `input.txt` contains three status lines.

### 5.5.2 Input File: `input.txt`

```
UART PASS
SPI PASS
I2C FAIL
```

### 5.5.3 Complete Source: `19_read_file.tcl`

```tcl
# Reading a file
# 1st created a sample file input.txt in folder
set filename "05_file_automation/input.txt"

# Open the file in read mode.
set file [open $filename r]

# read retrieves the complete file contents.
set data [read $file]

close $file

puts "File contents:"
puts $data

# Output:
# File contents:
# UART PASS
# SPI PASS
# I2C FAIL
```

### 5.5.4 Expected Console Output

```
File contents:
UART PASS
SPI PASS
I2C FAIL
```

### 5.5.5 Understanding Relative Paths

A relative path is resolved from Tcl's current working directory, which is generally the directory from which the Tcl interpreter was launched. It is not automatically resolved relative to the directory containing the script.

For example, when the command is run from the repository root:

```
tclsh .\05_file_automation\19_read_file.tcl
```

the script path `05_file_automation/input.txt` points to the input file in that subdirectory. Using only `input.txt` would make Tcl search for that file in the repository root, which was the cause of the earlier `couldn't open "input.txt"` error.

### 5.5.6 Important File-Handling Rule

Open a file in read mode only when it exists at the specified path, and close the channel after reading. The supplied example reads the entire file into one variable; line-by-line processing with `gets` is covered in the later pattern-matching phase.

## 5.6 20 — Writing Multiple Lines to a File

### 5.6.1 Concept

A file opened in write mode can receive multiple calls to `puts`. Each call writes one line to the file. This is useful for generating a report without manually editing a text file.

### 5.6.2 Complete Source: `20_write_file.tcl`

```tcl
# Writing multiple lines to a file

set filename "report.txt"

set file [open $filename w]

# puts can write text directly into an opened file.
puts $file "Tcl Automation Report"
puts $file "----------------------"
puts $file "UART : PASS"
puts $file "SPI  : PASS"
puts $file "I2C  : FAIL"

close $file

puts "Report written to: $filename"

# Output:
# Report written to: report.txt
```

### 5.6.3 Expected Console Output

```
Report written to: report.txt
```

### 5.6.4 Generated Report Contents

The script writes the following lines to `report.txt`:

```
Tcl Automation Report
---------------------
UART : PASS
SPI  : PASS
I2C  : FAIL
```

### 5.6.5 Why This Is Useful

Scripts can generate repeatable, consistently formatted reports from results collected during a tool run. The same pattern can be adapted to write simulation status, module lists, or other summary information.

## 5.7 21 — Directory Operations

### 5.7.1 Concept

The `file exists` command checks whether a file-system path exists. The `file mkdir` command creates a directory. The `glob` command retrieves items matching a pattern; the option `-nocomplain` allows an empty match result without raising the usual no-match error.

### 5.7.2 Complete Source: `21_directory_operations.tcl`

```tcl
# Basic directory operations

set directory "reports"

# file exists checks whether a file or directory exists.
if {![file exists $directory]} {
    # file mkdir creates the directory.
    file mkdir $directory
    puts "Directory created: $directory"
} else {
    puts "Directory already exists: $directory"
}

# glob returns files or directories matching a pattern.
set files [glob -nocomplain -directory $directory *]

puts "Items in directory: [llength $files]"

# Output:
# Directory created: reports
# Items in directory: 0
```

### 5.7.3 Expected Output on the First Run

```
Directory created: reports
Items in directory: 0
```

### 5.7.4 Expected Output on Subsequent Runs

```
Directory already exists: reports
Items in directory: 0
```

### 5.7.5 Explanation

On the first run, the directory does not exist, so the script creates it and prints a creation message. On subsequent runs, the directory exists and the `else` branch prints a different message. Since the new directory contains no entries, the `glob` result is an empty list and `llength` returns zero.

### 5.7.6 A Note About `glob`

The pattern `*` matches entries inside the selected directory. The command uses `-directory $directory` to search in `reports`, rather than the current directory. The `-nocomplain` option makes the script robust when there are no matching entries.

## 5.8 22 — Generating a File Report

### 5.8.1 Concept

This exercise combines a Tcl list, a `foreach` loop, and file operations. It writes each module name to a file, closes the file, reopens it in read mode, and prints the contents.

### 5.8.2 Complete Source: `22_file_report.tcl`

```
# Generate a simple file report

set filename "modules.txt"

set modules {ALU FIFO UART SPI I2C}

# Open the file for writing.
set file [open $filename w]

# Write each module into the file.
foreach module $modules {
    puts $file $module
}

close $file

# Read the generated file.
set file [open $filename r]
set data [read $file]
close $file

puts "Module Report:"
puts $data
```

```
24
25 # Output:
26 # Module Report:
27 # ALU
28 # FIFO
29 # UART
30 # SPI
31 # I2C
```

### 5.8.3 Expected Console Output

```
Module Report:
ALU
FIFO
UART
SPI
I2C
```

### 5.8.4 Generated File Contents

The script creates `modules.txt`, with one module name per line:

```
ALU
FIFO
UART
SPI
I2C
```

### 5.8.5 Execution Flow

1. Store the module names in the list `modules`.
2. Open `modules.txt` in write mode.
3. Iterate over the list using `foreach`.
4. Write each module name on its own line.
5. Close the file after writing.
6. Reopen the same file in read mode and read its contents.
7. Close the file and display the report.

## 5.9 File Modes and Important Commands

### 5.9.1 File Modes

| Mode | Meaning in this phase |
|---|---|
| `r` | Open an existing file for reading. The operation fails if the file cannot be opened. |
| `w` | Open a file for writing; create it if necessary and replace existing contents. |

### 5.9.2 Command Reference

| Command | Purpose |
|---|---|
| `open` | Opens a file and returns a channel identifier. |
| `close` | Closes an opened channel and releases the resource. |
| `read` | Reads the available contents from an opened channel. |
| `puts` | Writes a line to the console or to a specified file channel. |
| `file exists` | Checks whether a path exists. |
| `file mkdir` | Creates a directory. |
| `glob` | Finds filesystem entries matching a pattern. |
| `llength` | Returns the number of elements in a list. |
| `foreach` | Iterates through the elements of a list. |
| `set` | Creates or updates a variable. |

## 5.10 Practical Automation and EDA Applications

The techniques in this phase can be adapted to several automation tasks:

- read synthesis or simulation reports from files;
- create output reports automatically;
- scan directories for Verilog or SystemVerilog source files;
- collect log files from tool runs;
- generate module lists and summary files;
- process multiple design files in an automated workflow.

These are applications of the demonstrated file-handling patterns. The supplied exercises themselves use simple text files and module names rather than invoking an EDA tool.

## 5.11 Common Issues and Good Practices

### 5.11.1 File Not Found

A read operation can fail if the filename is wrong, the file is absent, or the path is resolved from an unexpected working directory. Confirm the working directory and use the path appropriate to how the script is launched.

### 5.11.2 Always Close File Channels

After reading or writing, call `close` on the channel. This releases the resource and ensures that pending output is finalized.

### 5.11.3 Understand Write Mode

Opening a file using `w` overwrites its existing contents. Use this mode when generating a fresh output file, and be aware that previous contents will be replaced.

### 5.11.4 Expect Directory Output to Change

The directory exercise intentionally prints different messages depending on whether `reports` already exists. Its first-run output and subsequent-run output are both valid.

## 5.12 Review Exercises

### 5.12.1 Repository Exercises

1. Run `18_create_file.tcl`. Inspect `output.txt` and confirm the line written to it.
2. Run `19_read_file.tcl`. Explain why the script uses `05_file_automation/input.txt` for the supplied execution location.
3. Run `20_write_file.tcl`. Open `report.txt` and compare its contents with the source.
4. Run `21_directory_operations.tcl` twice. Explain why the first line differs on the second run.
5. Run `22_file_report.tcl`. Describe how `foreach` writes each module into `modules.txt`.

### 5.12.2 Review Questions

1. What does the `open` command return?
2. What is the difference between opening a file with `r` and `w`?
3. Why should a file channel be closed after use?
4. What does `read` do in the supplied example?
5. How does the current working directory affect relative paths?
6. What is the purpose of `file exists` and `file mkdir`?
7. Why is `-nocomplain` used with `glob`?
8. How do `foreach` and a file channel work together in the module-report exercise?
9. Why does the directory script produce different output on its first and later runs?
10. Name two ways these file-handling patterns could be applied in an EDA workflow.

## 5.13 Chapter Summary

Phase 05 introduces file creation, file reading, multi-line writing, directory management, and report generation. The five exercises demonstrate how Tcl scripts can interact with text files and the filesystem, and how lists and loops can be combined with file operations to automate repetitive tasks.

The most important practices are to choose the correct file mode, use paths that match the current working directory, and close every file channel after use. These skills provide a foundation for later phases that search files and parse logs and reports.

### Phase Completion Checklist

- ☐ Tcl-018: Create a file.
- ☐ Tcl-019: Read an existing file.
- ☐ Tcl-020: Write multiple lines to a file.
- ☐ Tcl-021: Create and inspect a directory.
- ☐ Tcl-022: Generate and read a module report.

**Next chapter:** Phase 06 — Pattern Matching.

# Chapter 6

# Pattern Matching

## 6.1 Introduction

Pattern matching helps a Tcl script discover files, identify patterns in text, search logs, and extract structured information. Automation tools often produce large amounts of text, so matching and extracting the important details is a core skill for reliable scripting.

This chapter follows the supplied `06_pattern_matching` folder and its four exercises:

- `23_glob_matching.tcl`
- `24_regular_expressions.tcl`
- `25_log_parser.tcl`
- `26_data_extractor.tcl`

The folder also contains `test.log`, which is used by the log parser.

## 6.2 Learning Objectives

After completing this chapter, you should be able to:

- find files with `glob` and use wildcard patterns;
- explain the purpose of `-nocomplain`;
- test text with `regexp` and use basic regular-expression syntax;
- capture matched fields into Tcl variables;
- read files one line at a time using `gets`;
- detect error lines and extract structured report values.

## 6.3 23 — Glob Matching

### 6.3.1 Concept

The `glob` command finds filesystem paths that match a pattern. It is commonly used to collect source files, log files, and reports before processing them. In a glob pattern, the `*` wildcard matches any sequence of characters.

| Pattern | Purpose |
|---|---|
| `*.tcl` | Match Tcl files in the searched directory |
| `*.log` | Match log files in the searched directory |
| `report_*.txt` | Match text files whose names begin with `report_` |

### 6.3.2 The `-nocomplain` Option

If no path matches a glob pattern, a normal `glob` call can raise an error. The `-nocomplain` option suppresses that no-match error and returns an empty list instead. This is useful when matching files are optional.

### 6.3.3 Complete Source: `23_glob_matching.tcl`

```tcl
# File matching using glob

# glob finds files or directories that match a pattern.
set files [glob -nocomplain "*.tcl"]

puts "Tcl files:"
puts $files

# Output:
# Tcl files:
# ...
```

### 6.3.4 Expected Output

The exact list depends on the current working directory and which `.tcl` files are present. The script prints the heading `Tcl files:` followed by the matched paths. If there are no matches, the second `puts` prints an empty value.

### 6.3.5 Working Directory Note

The pattern `"*.tcl"` is evaluated relative to Tcl's current working directory, unless a directory is included in the pattern or supplied through options. Therefore, running the script from the repository root may search a different directory than running it from inside `06_pattern_matching`. To search a specific folder, use a pattern such as `"06_pattern_matching/*.tcl"` or use `glob -directory`.

## 6.4 24 — Regular Expressions

### 6.4.1 Concept

The `regexp` command tests whether a regular-expression pattern matches text. It can also capture portions of a match into variables. Unlike `glob`, which is often used to find filesystem paths, `regexp` is designed for matching and extracting text.

### 6.4.2 Basic Pattern Elements

| Pattern | Meaning |
|---|---|
| `[A-Z]` | One uppercase English letter |
| `[a-z]` | One lowercase English letter |

| Pattern | Meaning |
|---|---|
| `[0-9]` | One digit |
| `+` | One or more occurrences of the preceding element |
| `*` | Zero or more occurrences of the preceding element |
| `.` | Any single character (subject to regular-expression rules) |
| `^` | Beginning of the string or line, depending on the pattern options |
| `$` | End of the string or line, depending on the pattern options |
| `(...)` | Capturing group |

### 6.4.3 Complete Source: `24_regular_expressions.tcl`

```
# Basic regular expression matching

set text "UART PASS"

# regexp checks whether a regular expression matches the text.
if {[regexp {UART} $text]} {
    puts "UART found"
}

# Extract a word using a regular expression.
regexp {([A-Z]+) ([A-Z]+)} $text -> protocol status

puts "Protocol: $protocol"
puts "Status: $status"

# Output:
# UART found
# Protocol: UART
# Status: PASS
```

### 6.4.4 Expected Output

```
UART found
Protocol: UART
Status: PASS
```

### 6.4.5 Capturing Groups and the `->` Variable

The pattern `([A-Z]+) ([A-Z]+)` matches two groups of uppercase letters separated by a space. The first group is assigned to `protocol` and the second to `status`. The special variable name `->` is used as the match variable for the complete match when that value is not needed by the script.

## 6.5 25 — Log Parser

### 6.5.1 Concept

A log parser scans a text file and reports lines that match a pattern. The exercise opens `test.log`, reads one line at a time with `gets`, and checks each line for the text `ERROR:`.

### 6.5.2 Input File: `test.log`

The supplied input file contains:

```
INFO: UART test started
PASS: UART communication successful
ERROR: SPI communication failed
INFO: Test completed
```

### 6.5.3 Complete Source: `25_log_parser.tcl`

```tcl
# Simple log parser
# 1st created a test.log in the folder to test
set filename "06_pattern_matching/test.log"

set file [open $filename r]

# Process the file one line at a time.
while {[gets $file line] >= 0} {

    # Check whether the line contains ERROR.
    if {[regexp {ERROR:} $line]} {
        puts "Error found: $line"
    }
}

close $file

# Output:
# Error found: ERROR: SPI communication failed
```

### 6.5.4 Expected Output

```
Error found: ERROR: SPI communication failed
```

### 6.5.5 How the Loop Works

1. `open $filename r` opens the log file in read mode.
2. `gets $file line` reads the next line and stores it in `line`.
3. The loop continues while `gets` returns a non-negative value.
4. `regexp {ERROR:} $line` tests the current line.

5. Matching lines are printed, and `close` releases the file channel after the loop finishes.

### 6.5.6 Path and Execution Note

The script uses the relative path `06_pattern_matching/test.log`. This path assumes the script is launched from the repository's parent directory. If you run the script from inside `06_pattern_matching`, use `test.log` instead. Relative paths are resolved from the current working directory, not automatically from the script's own location.

## 6.6 26 — Data Extractor

### 6.6.1 Concept

Structured text often contains key-value fields. Regular expressions can capture the value after a known key and assign it to a Tcl variable. The exercise extracts a module name and status from a single string.

### 6.6.2 Complete Source: `26_data_extractor.tcl`

```
# Extract structured information from text

set log "Module=UART Status=PASS"

# Extract values using regular expressions.
regexp {Module=([A-Za-z0-9_]+)} $log -> module
regexp {Status=([A-Za-z0-9_]+)} $log -> status

puts "Module: $module"
puts "Status: $status"

# Output:
# Module: UART
# Status: PASS
```

### 6.6.3 Expected Output

```
Module: UART
Status: PASS
```

### 6.6.4 Understanding the Pattern

The character class `[A-Za-z0-9_]` accepts uppercase letters, lowercase letters, digits, and underscores. The quantifier `+` requires one or more such characters. The capturing parentheses store the matched value in `module` or `status`.

### 6.6.5 A Practical Limitation

The supplied script assumes both fields exist and match the expected format. In a more defensive parser, check the return value of each `regexp` call before using the captured variables. This avoids relying on a capture when the input format is incomplete or unexpected.

## 6.7 Glob and Regular Expressions Compared

| Feature | `glob` | `regexp` |
|---|---|---|
| Main purpose | Match filesystem paths | Match and extract text |
| Example | `*.tcl` | `Module=([A-Za-z0-9_]+)` |
| Typical input | Filenames and directory names | Log lines, reports, configuration text |
| Typical output | List of matching paths | Boolean-like match result and optional captures |

## 6.8 Command Reference

| Command | Purpose |
|---|---|
| `glob` | Finds files or directories matching a path pattern |
| `regexp` | Tests a regular expression and can capture matched groups |
| `open` | Opens a file and returns a channel identifier |
| `gets` | Reads one line from an opened channel |
| `close` | Closes an open channel |
| `while` | Repeats a block while its condition is true |
| `puts` | Prints text to standard output or a channel |

## 6.9 Pattern-Matching Workflow

**Find Candidate Files with `glob`**

↓

**Open a File with `open`**

↓

**Read Each Line with `gets`**

↓

**Search or Capture with `regexp`**

↓

**Report the Relevant Information**

## 6.10 Practical EDA Automation Uses

Pattern matching can support EDA workflows by:

- discovering Verilog, Tcl, and log files;
- searching synthesis and simulation logs for errors;
- extracting module names and PASS/FAIL status;
- locating timing metrics such as WNS and TNS when the report format is known;
- collecting information from multiple generated reports;
- building simple log analyzers and report summaries.

For example, a parser may be designed to recognize `Module=UART`, `Status=PASS`, timing values such as `WNS = -0.25`, or an error line such as `ERROR: Cannot find cell`. Each pattern should be adapted to the actual format emitted by the tool.

## 6.11 Review Exercises

1. Run `23_glob_matching.tcl` from the repository root and from inside the phase folder. Compare the results.
2. Change the glob pattern to `"*.log"` and explain what it finds in each working directory.
3. Run `24_regular_expressions.tcl`. Identify the two capturing groups in the regular expression.
4. Add a different line containing `ERROR:` to `test.log` and rerun the log parser.
5. Change the log parser's pattern to search for `PASS:`. What lines are reported?
6. Change the module and status values in `26_data_extractor.tcl` and verify the captured output.
7. Explain when you would use `glob` and when you would use `regexp`.
8. Why should a robust parser check whether a regular-expression match succeeded before using captured values?

## 6.12 Phase 06 Summary

Phase 06 introduced file matching with `glob`, wildcard patterns, and the `-nocomplain` option. It then developed regular-expression matching and capture groups, line-by-line file reading with `gets`, log error detection, and extraction of structured fields. Together, these skills form a foundation for parsing tool output and building practical EDA automation utilities.

### Phase Completion Checklist

☐ Run `23_glob_matching.tcl` and inspect the matched paths.

☐ Run `24_regular_expressions.tcl` and verify both captured fields.

☐ Run `25_log_parser.tcl` and confirm the error line is reported.

☐ Run `26_data_extractor.tcl` and verify the module and status values.

**Next phase:** Phase 07 — Error Handling.

# Chapter 7

# Error Handling

## 7.1 Introduction

Error handling is the practice of detecting problems, reporting them clearly, and deciding whether a script should continue or stop. Tcl automation scripts often interact with files, configuration values, tool commands, and generated reports. Any of these operations may fail or receive unexpected input.

Phase 07 contains three exercises:

- `27_catch.tcl` — basic error handling with `catch`;
- `28_error_handling.tcl` — handling a failed file-open operation;
- `29_validation.tcl` — validating module and status values.

## 7.2 Learning Objectives

After completing this chapter, you should be able to:

- use `catch` to prevent a command error from immediately terminating a script;
- interpret the return code produced by `catch`;
- capture and print an error message;
- handle file-opening failures safely;
- validate string, list, and numeric values before using them;
- distinguish input validation from error handling;
- apply these techniques to more reliable EDA automation.

## 7.3 The `catch` Command

### 7.3.1 Purpose

The Tcl `catch` command executes a script and reports whether that script completed normally or generated an error. The command's result is returned as an integer. If a variable name is supplied as the second argument, Tcl stores the command's result or error message in that variable.

### 7.3.2 Syntax

```
catch script ?resultVarName?
```

| Return code | Meaning |
|---|---|
| 0 | The script completed without an error. |
| 1 | The script generated an error. |

The exercises in this phase use the basic success and error cases. Tcl's `catch` command can also return other codes for other Tcl completion conditions, but those are outside the scope of the supplied exercises.

## 7.4 27 — Basic Error Handling with `catch`

### 7.4.1 Objective

This script deliberately evaluates division by zero to demonstrate how `catch` captures an error instead of allowing it to stop the script at that command.

### 7.4.2 Complete Source: `27_catch.tcl`

```
# Basic error handling with catch

# catch executes a command and prevents an error from stopping the script.
set result [catch {expr {10 / 0}} error_message]

puts "Return code: $result"
puts "Error message: $error_message"

# Output:
# Return code: 1
# Error message: divide by zero
```

### 7.4.3 Expected Output

```
Return code: 1
Error message: divide by zero
```

### 7.4.4 Step-by-Step Explanation

1. `expr {10 / 0}` attempts to evaluate an arithmetic expression that causes a division-by-zero error.
2. `catch` executes that expression and captures the error condition.
3. The return code is stored in `result`; the error text is stored in `error_message`.
4. The two `puts` commands display the return code and captured message.

The important point is that the script can inspect the failure and choose what to do next. The error has not disappeared: it has been captured so that the script can handle it.

## 7.5 28 — Handling a File-Operation Error

### 7.5.1 Objective

File operations can fail when a file is missing, a path is incorrect, or access is denied. The second exercise attempts to open a file named `missing_file.txt`. If opening it fails, the script prints the captured error and then continues to a final message.

### 7.5.2 Complete Source: `28_error_handling.tcl`

```
# Handling errors using catch

set filename "missing_file.txt"

# Try to open the file without stopping the script if it fails.
set status [catch {open $filename r} file]

if {$status == 0} {
    puts "File opened successfully"
    close $file
} else {
    puts "Error opening file: $file"
}

puts "Script continues running."

# Output:
# Error opening file: couldn't open "missing_file.txt": no such file or directory
# Script continues running.
```

### 7.5.3 Expected Output

When `missing_file.txt` does not exist at the current working directory, the expected output is:

```
Error opening file: couldn't open "missing_file.txt": no such file or directory
Script continues running.
```

The exact wording of operating-system file errors can vary between environments. The output above is the example documented for this exercise.

### 7.5.4 Understanding the Conditional

- `open $filename r` attempts to open the named file in read mode.
- If `catch` returns `0`, the open operation succeeded and `file` contains the channel identifier.
- If `catch` returns a nonzero error code, `file` contains the error message for this example.
- `close $file` is executed only after a successful open.
- The final `puts` runs after either branch, showing that the handled failure did not stop this script.

### 7.5.5 Why the File Is Closed Only on Success

A file channel is available only if the open operation succeeds. Closing it inside the success branch avoids attempting to close a channel when the variable contains an error message rather than an open channel identifier.

### 7.5.6 Relative-Path Note

The filename `missing_file.txt` is relative to Tcl's current working directory. If a file with that name exists in the directory from which Tcl is launched, the open operation may succeed and the output will differ. The documented error output assumes the file is absent from that working directory.

## 7.6 29 — Input Validation

### 7.6.1 Objective

Input validation checks that values meet expectations before the script uses them. The supplied example validates that the module name is not empty and that the status is one of the accepted values, `PASS` or `FAIL`.

### 7.6.2 Complete Source: `29_validation.tcl`

```tcl
# Basic input validation

set module "UART"
set status "PASS"

# Validate that the module name is not empty.
if {$module eq ""} {
    puts "Error: Module name is empty"
} else {
    puts "Module: $module"
}

# Validate the status value.
if {$status ni {PASS FAIL}} {
    puts "Error: Invalid status"
} else {
    puts "Status: $status"
}

# Output:
# Module: UART
# Status: PASS
```

### 7.6.3 Expected Output

```
Module: UART
Status: PASS
```

### 7.6.4 Checking for an Empty String

The expression `$module eq ""` checks whether the module variable contains an empty string. If it does, the script prints an error message; otherwise, it prints the module name.

The `eq` operator performs a string comparison. It is appropriate here because the value being checked is text.

### 7.6.5 Checking an Allowed Status List

The expression `$status ni {PASS FAIL}` checks whether the status value is not an element of the list `PASS FAIL`. If the value is not in that list, the script reports an invalid status. Otherwise, it prints the status.

For this exercise, the accepted status values are exactly `PASS` and `FAIL`.

## 7.7 String, List, and Numeric Validation

### 7.7.1 String Comparison Operators

| Operator | Meaning |
|---|---|
| `eq` | True when two strings are equal. |
| `ne` | True when two strings are not equal. |

Example:

```
set status "PASS"

if {$status eq "PASS"} {
    puts "Test passed"
}
```

### 7.7.2 List-Based Validation

The `ni` operator means “not in”. It checks whether a value is absent from a list. In the supplied validation exercise, `$status ni {PASS FAIL}` rejects any status other than the two accepted values.

```
set status "PASS"

if {$status ni {PASS FAIL}} {
    puts "Error: Invalid status"
} else {
    puts "Status: $status"
}
```

### 7.7.3 Numeric Comparison Operators

Numeric validation can use comparison operators to check whether a number lies within an expected range or meets a condition.

| Operator | Meaning |
|---|---|
| == | Equal |
| != | Not equal |
| > | Greater than |
| < | Less than |
| >= | Greater than or equal to |
| <= | Less than or equal to |

Example from the phase overview:

```
set value 10

if {$value < 0} {
    puts "Error: Invalid value"
}
```

This condition reports an error when `value` is less than zero. It does not print an error for the example value `10`.

## 7.8 Error Handling Versus Input Validation

These techniques solve related but different problems.

| Aspect | Input validation | Error handling with `catch` |
|---|---|---|
| Purpose | Check whether a value is acceptable | Capture an error raised by a command |
| When used | Before or during decisions about input | Around an operation that may fail |
| Example | Reject a status other than `PASS` or `FAIL` | Capture failure while opening a file |
| Outcome | The script decides whether the input meets a rule | The script receives a return code and result/error text |

Validation does not replace `catch`. A value can be valid while an operation using it still fails. Likewise, `catch` does not automatically decide whether a value is meaningful. Robust scripts often validate inputs and use `catch` around operations that may generate errors.

## 7.9 A General Error-Handling Workflow

**Receive an Input or Configuration Value**

↓

**Validate the Value**

↓

**Perform the Required Operation**

↓

**Capture Possible Errors with `catch`**

↓

**Report the Result or Handle the Failure**

↓

**Continue or Stop in a Controlled Way**

This is a general workflow described by the phase material. The appropriate response to an error depends on the task: some scripts can continue, while others should stop after reporting a critical failure.

## 7.10 Important Commands and Operators

| Command / operator | Purpose |
|---|---|
| `catch` | Executes a script and captures its completion code and result or error message. |
| `if` | Selects a branch based on a condition. |
| `eq` | Compares strings for equality. |
| `ne` | Compares strings for inequality. |
| `ni` | Tests whether a value is not present in a list. |
| `==` | Numeric equality comparison. |
| `!=` | Numeric inequality comparison. |
| `>` | Greater-than comparison. |
| `<` | Less-than comparison. |
| `>=` | Greater-than-or-equal comparison. |
| `<=` | Less-than-or-equal comparison. |
| `open` | Opens a file and returns a channel identifier if successful. |
| `close` | Closes an open file channel. |
| `puts` | Prints a message. |

## 7.11 Practical EDA Automation Applications

Error handling and validation are useful in scripts that automate simulation, synthesis, report collection, and other EDA tasks. Examples include:

- reporting a missing report file instead of allowing an unhandled file error;
- checking that a module name or configuration field is not empty;

- accepting only known result labels such as `PASS` and `FAIL`;
- capturing errors from Tcl commands that interact with files or tools;
- reporting unexpected input or failed operations in a consistent way;
- deciding whether a flow should continue or stop after a failure.

For example, a report-processing script could validate a required report filename, attempt to open the report inside `catch`, and print a useful diagnostic if the file cannot be opened. This is an application of the concepts in this chapter; it is not an additional script supplied in the phase folder.

## 7.12 Common Mistakes and Checks

- **Ignoring the return code:** inspect the value returned by `catch` to distinguish success from failure.
- **Using a file channel after a failed open:** only call `close` when the open operation succeeded.
- **Confusing validation with error handling:** validation checks a rule; `catch` captures an operation's error.
- **Using the wrong comparison:** use string comparisons such as `eq` and `ne` for text labels.
- **Assuming a file is missing everywhere:** relative paths are interpreted from the current working directory, so the result depends on where the script is launched.
- **Providing no useful diagnostic:** include enough context in an error message to help identify the failed operation.

## 7.13 Review Exercises

1. Run `27_catch.tcl`. Explain why the return code is `1`.
2. In `27_catch.tcl`, identify the variable that receives the error message.
3. Run `28_error_handling.tcl` in a directory where `missing_file.txt` does not exist. Explain why the final message is printed.
4. Create a file named `missing_file.txt` in the current working directory and rerun the second script. Explain why its output may change.
5. In `29_validation.tcl`, change the module value to an empty string and observe the message.
6. Change the status to a value other than `PASS` or `FAIL`. Explain which branch executes.
7. Explain the difference between `eq`, `ne`, and `ni`.
8. Write a numeric condition that reports an error when a value is negative.
9. Describe one EDA automation scenario in which validation and `catch` should be used together.

## 7.14 Phase 07 Summary

This phase introduced the basic techniques needed to make Tcl scripts more resilient. The `catch` command captures command errors and provides a return code and result/error message. File handling demonstrated how to branch safely after an open attempt. Validation exercises showed how to check empty strings, allowed status values, and numeric conditions.

These concepts are foundational for dependable automation: validate values before relying on them, handle operations that may fail, report useful messages, and make a deliberate decision about whether to continue.

### Phase Completion Checklist

- ☐ Run `27_catch.tcl` and verify the return code and error message.
- ☐ Run `28_error_handling.tcl` with the expected missing file and observe continued execution.
- ☐ Run `29_validation.tcl` with valid values.
- ☐ Test an empty module name and an invalid status value.
- ☐ Explain when to use input validation and when to use `catch`.

**Next phase:** Phase 08 — Automation Utilities.

# Chapter 8

# Automation Utilities

## 8.1 Introduction

Automation utilities let Tcl scripts interact with the operating system, read environment variables, accept command-line inputs, and combine several operations into a reusable workflow. Phase 08 moves from individual Tcl commands toward practical automation scripts.

The supplied folder contains four scripts and a generated report:

- `30_command_execution.tcl`
- `31_environment_variables.tcl`
- `32_arguments.tcl`
- `33_automation_script.tcl`
- `report.txt`

## 8.2 Learning Objectives

After completing this chapter, you should be able to:

- execute external commands using `exec`;
- read operating-system environment variables through Tcl's `env` array;
- understand the `argc` and `argv` variables;
- retrieve individual command-line arguments using `lindex`;
- validate required arguments and use `exit` status codes;
- generate a text report from a Tcl script;
- combine these features into a simple automation workflow.

## 8.3 30 — Executing Operating-System Commands

### 8.3.1 Concept

The Tcl `exec` command launches an external operating-system command and returns its output. The exact command syntax depends on the operating system and the program being invoked.

### 8.3.2 Complete Source: `30_command_execution.tcl`

```tcl
# Executing an operating-system command

# exec runs an external command and returns its output.
set result [exec echo "Hello from Tcl"]

puts "Command output: $result"

set result [exec cmd /c echo "Hello from Windows"]
puts "Command output: $result"

# Output:
# Command output: Hello from Tcl
# Command output: "Hello from Windows"
```

### 8.3.3 Expected Output

The supplied source documents this output on Windows:

```
Command output: Hello from Tcl
Command output: "Hello from Windows"
```

The second command explicitly invokes the Windows command interpreter with `cmd /c`. Because this script contains Windows-specific command syntax, it should be run in a Windows environment or adapted for another operating system.

### 8.3.4 Step-by-Step Explanation

1. `exec echo "Hello from Tcl"` executes the external `echo` command.
2. The command substitution `[...]` runs the command and inserts its returned output.
3. The output is assigned to `result`.
4. `puts` prints the stored output with a label.
5. The second command runs `cmd /c echo ...` through the Windows command interpreter.

### 8.3.5 Important Consideration

An external command may fail, for example if the executable is missing or the command returns an error. Phase 07's `catch` command can be used to capture such failures and report them in a controlled way.

## 8.4 31 — Environment Variables

### 8.4.1 Concept

Tcl exposes process environment variables through a built-in array named `env`. Environment variables can hold information about the operating system, username, executable search paths, tool installation paths, and configuration.

### 8.4.2 Complete Source: `31_environment_variables.tcl`

```tcl
# Accessing environment variables

# env is a Tcl array containing environment variables.
puts "Operating System: $::env(OS)"

puts "User: $::env(USERNAME)"

# Output:
# Operating System: Windows_NT
# User: Bibhu
```

### 8.4.3 Expected Output

The supplied source records the following example output:

```
Operating System: Windows_NT
User: Bibhu
```

The username and available variables depend on the machine and the environment in which Tcl is launched. The output above is the example recorded in the supplied script, not a guaranteed value on every computer.

### 8.4.4 Understanding `$::env(NAME)`

- `env` is a Tcl array whose elements correspond to environment-variable names.
- `OS` and `USERNAME` are the two keys accessed by the supplied script.
- The `::` prefix refers to the global namespace.
- Accessing a variable that is not defined can produce a Tcl error, so reusable scripts should check whether the requested environment variable exists.

### 8.4.5 Environment Variables in EDA

Environment variables are often used to locate tool installations, libraries, project directories, or executable programs. Their names and availability vary by operating system and installation, so scripts should not assume every variable exists.

## 8.5 32 — Command-Line Arguments

### 8.5.1 Concept

Command-line arguments allow a script to receive values when it starts, rather than requiring those values to be hard-coded in the source.

Tcl provides two built-in variables:

| Variable | Purpose |
|---|---|
| `argc` | Number of command-line arguments supplied to the script. |
| `argv` | Tcl list containing the supplied command-line arguments. |

The script filename itself is not included in `argv`.

### 8.5.2 Complete Source: `32_arguments.tcl`

```tcl
# Reading command-line arguments

# argc contains the number of arguments.
puts "Number of arguments: $argc"

# argv contains all command-line arguments as a list.
puts "Arguments: $argv"

if {$argc > 0} {
    puts "First argument: [lindex $argv 0]"
}

# Example run:
# tclsh .\08_automation_utilities\32_arguments.tcl UART PASS
# Number of arguments: 2
# Arguments: UART PASS
# First argument: UART
#
# Run without arguments:
# Number of arguments: 0
# Arguments:
```

### 8.5.3 Running the Script

From the repository root in Windows PowerShell, the supplied example is:

```
tclsh .\08_automation_utilities\32_arguments.tcl UART PASS
```

### 8.5.4 Expected Output with Two Arguments

```
Number of arguments: 2
Arguments: UART PASS
First argument: UART
```

### 8.5.5 Expected Output with No Arguments

```
Number of arguments: 0
Arguments:
```

### 8.5.6 Accessing Individual Arguments

Since `argv` is a Tcl list, `lindex` retrieves an element by index. Tcl list indexes start at zero, so `lindex $argv 0` returns the first argument.

```
set module [lindex $argv 0]
set status [lindex $argv 1]

puts "Module: $module"
puts "Status: $status"
```

This snippet demonstrates how to retrieve two arguments; it is an explanatory example from the phase README, rather than the complete content of `32_arguments.tcl`.

## 8.6 33 — Complete Automation Script

### 8.6.1 Concept

The fourth script combines argument checking, input validation, file writing, and report generation. It requires exactly one command-line argument: a module name. If the number of arguments is incorrect or the module name is empty, it prints a diagnostic and exits with a nonzero status.

### 8.6.2 Complete Source: `33_automation_script.tcl`

```
# Simple automation script

# Check whether a module name was provided.
if {$argc != 1} {
    puts "Usage: tclsh 33_automation_script.tcl <module>"
    exit 1
}

set module [lindex $argv 0]

# Validate that the module name is not empty.
if {$module eq ""} {
    puts "Error: Module name cannot be empty"
    exit 1
}

set filename "08_automation_utilities/report.txt"

# Open the report file for writing.
set file [open $filename w]

puts $file "Module Report"
puts $file "-------------"
puts $file "Module: $module"
puts $file "Status: PASS"

close $file
```

```tcl

puts "Report generated: $filename"

# Example output:
# Report generated: 08_automation_utilities/report.txt
```

### 8.6.3 How the Script Works

1. **Check argument count.** The condition `$argc != 1` rejects calls that do not provide exactly one argument.
2. **Read the module name.** `lindex $argv 0` retrieves the first argument.
3. **Validate the module.** The condition `$module eq ""` rejects an empty module name.
4. **Choose the report path.** The script sets `filename` to `08_automation_utilities/report.txt`.
5. **Open the report.** The mode `w` opens the file for writing and creates it if needed; existing content is overwritten.
6. **Write report lines.** Each `puts $file ...` writes a line to the file channel.
7. **Close the file.** `close $file` closes the report channel.
8. **Report completion.** The final `puts` prints the report path.

### 8.6.4 Running the Script

Run from the repository root in Windows PowerShell:

```
tclsh .\08_automation_utilities\33_automation_script.tcl UART
```

### 8.6.5 Expected Console Output

```
Report generated: 08_automation_utilities/report.txt
```

### 8.6.6 Generated File: `report.txt`

The supplied archive contains this report:

```
Module Report
-------------
Module: UART
Status: PASS
```

The generated report content depends on the module argument supplied when the script is run. The provided file shows the example module `UART`.

### 8.6.7 Argument Validation Examples

If the script is run without a module argument, it prints the usage message and exits with status `1`. If an empty argument is supplied, it prints the empty-module error and exits with status `1`.

```
Usage: tclsh 33_automation_script.tcl <module>
```

The usage line is shown when the argument count is not exactly one. The exact console output for a particular run depends on the arguments supplied.

### 8.6.8 Relative-Path Consideration

The report path `08_automation_utilities/report.txt` is relative to the current working directory. The example command assumes the script is launched from the repository root. If the script is launched from inside the `08_automation_utilities` directory, that path may refer to a nested location that does not exist. Run from the expected directory or adjust the path to match the launch location.

## 8.7 The `exit` Command and Exit Status

### 8.7.1 Purpose

The `exit` command terminates a Tcl script and supplies an exit status to the calling environment.

| Command | Conventional meaning |
|---|---|
| `exit 0` | Successful completion. |
| `exit 1` | Error or unsuccessful execution. |

The phase's automation script uses `exit 1` when its required arguments are invalid. A caller, shell, or larger automation system can use the exit status to decide whether a task succeeded.

## 8.8 Automation Workflow

A general automation script often follows this sequence:

**Read Command-Line Arguments**
↓
**Validate Required Inputs**
↓
**Run the Required Operation**
↓
**Process the Result**
↓
**Generate a Report**
↓
**Return an Appropriate Exit Status**

The supplied Phase 08 scripts demonstrate individual pieces of this workflow. The fourth script combines arguments, validation, and report generation.

## 8.9 Command and Variable Reference

| Command / variable | Purpose |
|---|---|
| `exec` | Executes an external operating-system command and returns its output. |
| `env` | Tcl array that exposes process environment variables. |
| `argc` | Number of arguments supplied to the script. |
| `argv` | List of command-line arguments. |
| `lindex` | Retrieves an item from a Tcl list by index. |
| `exit` | Terminates the script with an exit status. |
| `open` | Opens a file and returns a channel identifier. |
| `puts` | Writes text to the console or a file channel. |
| `close` | Closes an open file channel. |
| `if` | Selects a branch based on a condition. |
| `eq` | Tests string equality. |
| `catch` | Captures errors from a Tcl script or command. |

## 8.10 Practical EDA Automation Applications

The techniques in this phase can support EDA workflows by:

- passing a module or design name to a script at launch time;
- using environment variables to locate tools and project resources;
- launching supported external commands;
- checking required arguments before starting a flow;
- generating concise text reports;
- combining file operations, pattern matching, and error handling from earlier phases.

A possible workflow is to accept a design name, run a tool command, process its output, and create a summary report. The specific tool commands and report formats must be adapted to the EDA environment being used.

## 8.11 Common Mistakes and Checks

- **Assuming all operating systems use the same command syntax:** `cmd /c` is Windows-specific.
- **Assuming an environment variable always exists:** available names vary by machine and configuration.
- **Confusing `argc` and `argv`:** `argc` is a count; `argv` is the list of values.
- **Using the wrong argument index:** the first argument is at index zero.
- **Skipping validation:** check required arguments before using them.

- **Overwriting a report unintentionally:** opening a file with mode `w` replaces its existing contents.
- **Using an unexpected relative path:** relative paths are resolved from the current working directory.
- **Ignoring command failures:** use `catch` when an external or Tcl operation may fail and the script needs controlled error handling.

## 8.12 Review Exercises

1. Run `30_command_execution.tcl` on Windows and identify the purpose of `exec`.
2. Explain why the second command in `30_command_execution.tcl` uses `cmd /c`.
3. Run `31_environment_variables.tcl`. Which environment-variable keys does it read?
4. Explain what may happen if an environment variable used by a script is not defined.
5. Run `32_arguments.tcl` with `UART PASS`. Compare the values of `argc` and `argv`.
6. Run `32_arguments.tcl` without arguments. Why is the first-argument message not printed?
7. Run `33_automation_script.tcl` with a module name and inspect `report.txt`.
8. Run the fourth script without an argument. Explain the purpose of `exit 1`.
9. Change the module argument from `UART` to another name and inspect the generated report.
10. Explain why the report file may be written to an unexpected location if the script is launched from a different working directory.

## 8.13 Phase 08 Summary

Phase 08 introduced the practical Tcl utilities needed to connect scripts to the operating system and make them reusable. The `exec` command runs external commands, `env` exposes environment variables, and `argc` and `argv` provide command-line input. The `lindex` command retrieves individual arguments, while validation and `exit` allow a script to reject invalid input. Finally, the report-generation exercise combines argument handling and file writing into a small automation workflow.

### Phase Completion Checklist

- ☐ Run `30_command_execution.tcl` and inspect its output.
- ☐ Run `31_environment_variables.tcl` and identify the accessed environment variables.
- ☐ Run `32_arguments.tcl` with and without arguments.
- ☐ Run `33_automation_script.tcl` with a module name and inspect the generated report.
- ☐ Test the argument-count error path.
- ☐ Explain the relationship between input validation, report generation, and exit status.

**Next phase:** Phase 09 — EDA Workflows.

# Chapter 9

# EDA Workflows

## 9.1 Introduction

Phase 09 brings together Tcl skills developed in Phases 01–08 and applies them to EDA-style automation. The scripts demonstrate how to inventory project files, count log messages, extract timing metrics, summarize cell counts, and generate a design-specific summary report.

The supplied folder contains five exercises:

- `34_file_reporter.tcl`
- `35_log_analyzer.tcl`
- `36_timing_report_parser.tcl`
- `37_cell_statistics.tcl`
- `38_eda_automation_flow.tcl`

It also contains sample inputs such as `sample.log`, `timing.rpt`, `cells.rpt`, and the example `UART_summary.rpt`.

## 9.2 Learning Objectives

By completing this chapter, you should be able to:

- discover files with `glob` and display their names;
- process logs one line at a time and count message categories;
- use regular expressions to extract numeric report values;
- split structured report lines and calculate totals;
- validate command-line arguments and handle file-opening errors;
- generate a summary report from parsed timing information;
- combine file operations, parsing, validation, and reporting into a workflow.

## 9.3 34 — EDA File Reporter

### 9.3.1 Concept

A file reporter searches a selected directory, counts matching Tcl files, and prints each matching filename. This is useful when a project contains many scripts or source files and a quick inventory is needed.

### 9.3.2 Complete Source: `34_file_reporter.tcl`

```tcl
# EDA file reporter

set directory "09_eda_workflows"

# Find Tcl files in the selected directory.
set files [glob -nocomplain -directory $directory *.tcl]

puts "EDA File Report"
puts "---------------"
puts "Directory: $directory"
puts "Tcl files: [llength $files]"

# Display each matching file.
foreach file $files {
    puts [file tail $file]
}
```

### 9.3.3 Expected Output

When run from the repository root and the folder contains the five supplied scripts, the output is:

```
EDA File Report
---------------
Directory: 09_eda_workflows
Tcl files: 5
34_file_reporter.tcl
35_log_analyzer.tcl
36_timing_report_parser.tcl
37_cell_statistics.tcl
38_eda_automation_flow.tcl
```

### 9.3.4 Step-by-Step Explanation

1. `directory` stores the folder path to scan.
2. `glob -nocomplain -directory $directory *.tcl` searches that folder for files matching `*.tcl`.
3. `llength $files` counts the matching paths.
4. `foreach` iterates over each path in the list.
5. `file tail $file` returns only the final filename component.

### 9.3.5 Important Commands

| Command | Purpose |
|---|---|
| `glob` | Finds paths matching a pattern. |
| `-nocomplain` | Prevents a no-match glob from raising an error. |
| `-directory` | Specifies the directory in which to search. |
| `llength` | Returns the number of items in a list. |
| `foreach` | Runs a block for each item in a list. |
| `file tail` | Returns the last component of a path. |

### 9.3.6 EDA Applications

File discovery can be used to inventory RTL files, constraint files, Tcl scripts, generated reports, or other project resources. This exercise specifically lists Tcl files.

## 9.4 35 — EDA Log Analyzer

### 9.4.1 Concept

Tool logs often contain status labels such as `PASS`, `FAIL`, `WARNING`, and `ERROR`. This script reads a log one line at a time and counts how many lines contain each label.

### 9.4.2 Input File: `sample.log`

```
INFO: Synthesis started
PASS: RTL compilation successful
WARNING: Unconnected signal detected
PASS: Synthesis completed
ERROR: Timing constraint missing
INFO: Flow completed
```

### 9.4.3 Complete Source: `35_log_analyzer.tcl`

```
# EDA log analyzer
# 1st created a file sample.log for test

set filename "09_eda_workflows/sample.log"

set pass_count 0
set fail_count 0
set warning_count 0
set error_count 0

set file [open $filename r]

# Process the log one line at a time.
while {[gets $file line] >= 0} {

    if {[regexp {PASS:} $line]} {
        incr pass_count
```

```
    } elseif {[regexp {FAIL:} $line]} {
        incr fail_count
    } elseif {[regexp {WARNING:} $line]} {
        incr warning_count
    } elseif {[regexp {ERROR:} $line]} {
        incr error_count
    }
}

close $file

puts "EDA Log Analysis"
puts "----------------"
puts "PASS: $pass_count"
puts "FAIL: $fail_count"
puts "WARNING: $warning_count"
puts "ERROR: $error_count"
```

### 9.4.4 Expected Output

```
EDA Log Analysis
----------------
PASS: 2
FAIL: 0
WARNING: 1
ERROR: 1
```

### 9.4.5 How the Parser Works

1. Each counter is initialized to zero before the loop.
2. `open $filename r` opens the input log in read mode.
3. `gets` reads a line and stores it in `line`.
4. The `regexp` conditions check for each category.
5. `incr` increases the corresponding counter when a category matches.
6. `close` closes the file after all lines have been processed.

Because the checks use an `if`/`elseif` chain, each line increments at most one category counter: the first matching category wins. Lines containing none of these labels are ignored.

### 9.4.6 EDA Applications

The same pattern can be adapted to simulation, synthesis, implementation, or regression logs. The matching strings should be aligned with the actual log format produced by the tool.

## 9.5 36 — Timing Report Parser

### 9.5.1 Concept

Timing reports contain metrics that can be extracted from text and checked against a design's timing requirements. This exercise extracts WNS, TNS, and a violation count from `timing.rpt`, then produces a simplified PASS/FAIL assessment.

### 9.5.2 Input File: `timing.rpt`

```
WNS: -0.25
TNS: -10.40
VIOLATIONS: 3
```

### 9.5.3 Complete Source: `36_timing_report_parser.tcl`

```tcl
# Simple timing report parser
# 1st created a sample timing.rpt for test

set filename "09_eda_workflows/timing.rpt"

set wns 0
set tns 0
set violations 0

set file [open $filename r]

# Read and extract timing information.
while {[gets $file line] >= 0} {

    if {[regexp {WNS:\s+(-?[0-9.]+)} $line -> value]} {
        set wns $value
    }

    if {[regexp {TNS:\s+(-?[0-9.]+)} $line -> value]} {
        set tns $value
    }

    if {[regexp {VIOLATIONS:\s+([0-9]+)} $line -> value]} {
        set violations $value
    }
}

close $file

puts "Timing Report"
puts "-------------"
puts "WNS: $wns"
puts "TNS: $tns"
puts "Violations: $violations"
```

```tcl

# Check timing status.
if {$wns < 0 || $violations > 0} {
    puts "Timing Status: FAIL"
} else {
    puts "Timing Status: PASS"
}
```

### 9.5.4 Expected Output

```
Timing Report
-------------
WNS: -0.25
TNS: -10.40
Violations: 3
Timing Status: FAIL
```

### 9.5.5 Regular Expression Breakdown

| Pattern element | Meaning |
|---|---|
| `\s+` | One or more whitespace characters. |
| `-?` | Optional minus sign. |
| `[0-9.]+` | One or more digits or decimal points. |
| `([0-9]+)` | Captures one or more digits, used for the violation count. |
| `(...)` | Creates a capture group whose value is assigned to `value`. |

The expression `regexp {WNS:\s+(-?[0-9.]+)} $line -> value` matches a WNS field and captures the numeric-looking text after it. The variable `->` receives the complete match, while `value` receives the capture group.

### 9.5.6 Timing Metrics

| Metric | Meaning |
|---|---|
| `WNS` | Worst Negative Slack. |
| `TNS` | Total Negative Slack. |
| `VIOLATIONS` | Number of reported violations in this sample report. |

### 9.5.7 Status Rule Used by the Exercise

The script reports `FAIL` when `WNS` is negative or the violation count is greater than zero. Otherwise, it reports `PASS`. In the supplied report, WNS is `-0.25` and the violation count is `3`, so the result is `FAIL`.

This is a simplified educational check. Real sign-off criteria should follow the design's constraints, the tool's report definitions, and the project's timing requirements. The parser also

initializes metrics to zero, so a missing or unrecognized field can be mistaken for a valid zero unless the script is extended to check that every required field was found.

## 9.6 37 — Cell Statistics

### 9.6.1 Concept

A cell statistics report can list cell types and their counts. The script splits each line into fields, prints the cell type and count, and accumulates the number of rows and the total number of cells.

### 9.6.2 Complete Source: `37_cell_statistics.tcl`

```tcl
# Cell statistics analyzer
# 1st created a sample cells.rpt for test

set filename "09_eda_workflows/cells.rpt"

set total_cells 0
set cell_count 0

set file [open $filename r]

# Read each cell type and its count.
while {[gets $file line] >= 0} {

    set fields [split $line]

    set cell_type [lindex $fields 0]
    set count [lindex $fields 1]

    puts "$cell_type : $count"

    incr total_cells $count
    incr cell_count
}

close $file

puts "----------------"
puts "Cell types: $cell_count"
puts "Total cells: $total_cells"
```

### 9.6.3 Example Input Format

The README and the script's output comments illustrate a report in this format:

```
INV_X1 120
NAND2_X1 80
NOR2_X1 60
DFF_X1 40
```

```
BUF_X1 100
```

### 9.6.4 Illustrative Output for the Example Input

```
INV_X1 : 120
NAND2_X1 : 80
NOR2_X1 : 60
DFF_X1 : 40
BUF_X1 : 100
----------------
Cell types: 5
Total cells: 400
```

### 9.6.5 Important Note About the Supplied Archive

The `cells.rpt` file in the uploaded archive is empty. Therefore, running `37_cell_statistics.tcl` against that file as supplied will not produce the illustrative five cell rows; it will print the separator, `Cell types: 0`, and `Total cells: 0`. The output above is the intended example documented in the source comments and README, assuming the example input lines have been placed into `cells.rpt`.

### 9.6.6 Step-by-Step Explanation

1. `split $line` separates each input line into a Tcl list of fields.
2. `lindex $fields 0` extracts the cell type.
3. `lindex $fields 1` extracts the count.
4. `incr total_cells $count` adds the count to the running total.
5. `incr cell_count` increments the number of processed rows by one.

### 9.6.7 Assumptions and Limitations

The script assumes that each non-empty line contains at least two fields and that the second field is an integer. It does not skip blank lines, comments, or malformed rows. A robust production parser should validate the field count and numeric value before updating totals.

## 9.7 38 — Complete EDA Automation Flow

### 9.7.1 Concept

The final script combines command-line argument handling, validation, file access, regular-expression parsing, conditional status checking, and report generation. It accepts one design name, reads the timing report, calculates a simplified timing status, and writes a design-specific summary report.

### 9.7.2 Complete Source: `38_eda_automation_flow.tcl`

```tcl
# Complete EDA automation flow

# Check command-line arguments.
if {$argc != 1} {
    puts "Usage: tclsh 38_eda_automation_flow.tcl <design>"
    exit 1
}

set design [lindex $argv 0]

# Validate the design name.
if {$design eq ""} {
    puts "Error: Design name cannot be empty"
    exit 1
}

set timing_file "09_eda_workflows/timing.rpt"
set report_file "09_eda_workflows/${design}_summary.rpt"

set wns 0
set violations 0

# Safely open the timing report.
set status [catch {open $timing_file r} file]

if {$status != 0} {
    puts "Error opening timing report: $file"
    exit 1
}

# Parse the timing report.
while {[gets $file line] >= 0} {

    if {[regexp {WNS:\s+(-?[0-9.]+)} $line -> value]} {
        set wns $value
    }

    if {[regexp {VIOLATIONS:\s+([0-9]+)} $line -> value]} {
        set violations $value
    }
}

close $file

# Determine timing status.
if {$wns < 0 || $violations > 0} {
    set timing_status "FAIL"
} else {
    set timing_status "PASS"
}
```

```tcl

# Generate summary report.
set report [open $report_file w]

puts $report "EDA Automation Summary"
puts $report "----------------------"
puts $report "Design: $design"
puts $report "WNS: $wns"
puts $report "Violations: $violations"
puts $report "Timing Status: $timing_status"

close $report

puts "EDA flow completed."
puts "Summary report: $report_file"
```

### 9.7.3 Running the Script

From the repository root, the example invocation is:

```
tclsh .\09_eda_workflows\38_eda_automation_flow.tcl UART
```

### 9.7.4 Expected Console Output

```
EDA flow completed.
Summary report: 09_eda_workflows/UART_summary.rpt
```

### 9.7.5 Generated Summary: `UART_summary.rpt`

```
EDA Automation Summary
----------------------
Design: UART
WNS: -0.25
Violations: 3
Timing Status: FAIL
```

This matches the supplied example report. If a different design name is passed, the report filename and the `Design:` line change accordingly.

### 9.7.6 Execution Stages

1. **Argument check:** the script requires exactly one argument.
2. **Input validation:** it rejects an empty design name.
3. **Input file access:** `catch` captures an error if opening the timing report fails.
4. **Parsing:** regular expressions extract WNS and the violation count.
5. **Decision:** the script selects `PASS` or `FAIL` using the specified rule.
6. **Report generation:** the summary is written to a file named using the design argument.

7. **Completion message:** the console displays the generated report path.

### 9.7.7 Path Consideration

The input and output paths are relative to the current working directory. The example command assumes that Tcl is launched from the repository root. Launching the script from inside `09_eda_workflows` without changing the paths may cause it to look for a nested `09_eda_workflows` directory.

### 9.7.8 Robustness Considerations

The script safely handles failure to open the input timing report, but it does not use `catch` around opening the output report. It also initializes parsed values to zero and does not check whether each required field was found. These are reasonable opportunities for future improvements when moving from a learning exercise to production automation.

## 9.8 Workflow Architecture

The scripts in this phase build toward a general EDA data-processing workflow:

**Discover or Receive Input Files**
↓
**Read Logs and Reports**
↓
**Parse Text and Extract Values**
↓
**Calculate Statistics or Check Conditions**
↓
**Determine a Result**
↓
**Generate a Summary Report**

## 9.9 Command Reference

| Command / variable | Purpose |
|---|---|
| `glob` | Finds files matching a path pattern. |
| `file tail` | Extracts the filename from a path. |
| `llength` | Counts elements in a list. |
| `foreach` | Iterates over list elements. |
| `open` | Opens a file and returns a channel identifier. |
| `gets` | Reads one line from a channel. |
| `close` | Closes a channel. |
| `regexp` | Matches text and captures fields. |
| `split` | Splits a string into fields. |
| `lindex` | Retrieves an element from a list. |
| `incr` | Increments a numeric variable. |

| Command / variable | Purpose |
|---|---|
| `catch` | Captures command errors. |
| `argc` | Number of command-line arguments. |
| `argv` | List of command-line arguments. |
| `exit` | Terminates the script with an exit status. |
| `puts` | Prints text or writes it to a channel. |

## 9.10 Practical EDA Applications

The examples in this phase illustrate automation patterns that can be adapted for:

- RTL and project-file inventories;
- simulation, synthesis, and implementation log summaries;
- timing metric extraction from reports;
- cell-count and gate-statistics processing;
- automated PASS/FAIL checks;
- report generation for multiple designs;
- repeatable regression and build workflows.

The scripts use sample files and simplified report formats. Real tool reports may have different labels, units, section structures, or timing criteria. Always adapt and validate parsing expressions against the report format and project requirements.

## 9.11 Common Mistakes and Checks

- **Incorrect working directory:** relative paths depend on where Tcl is launched.
- **Assuming sample data exists:** check whether an input report is populated before interpreting its output.
- **Malformed report lines:** validate fields before treating extracted text as a number.
- **Missing metrics:** do not assume a default zero means a metric was successfully found.
- **Overly broad regular expressions:** adapt patterns to the exact report syntax.
- **Simplified timing checks:** use project sign-off requirements rather than treating this exercise's rule as universal.
- **Unprotected output operations:** consider handling failures when creating summary reports.

## 9.12 Review Exercises

1. Run `34_file_reporter.tcl`. Explain the role of `glob`, `llength`, and `file tail`.
2. Add another Tcl file to the folder and rerun the file reporter. How does the count change?

3. Run `35_log_analyzer.tcl`. Explain how each counter is updated.
4. Add a `FAIL:` line to `sample.log` and rerun the log analyzer.
5. Run `36_timing_report_parser.tcl`. Identify the captured WNS, TNS, and violation values.
6. Change WNS to a non-negative value and set violations to zero. What status does the script report?
7. Populate `cells.rpt` with the example cell rows, then run `37_cell_statistics.tcl`. Verify the row count and total.
8. Run `38_eda_automation_flow.tcl` with `UART`. Inspect the generated summary.
9. Run the final script without an argument. Explain the usage message and exit status.
10. Identify two improvements needed before adapting the timing parser for a production sign-off flow.

## 9.13 Phase 09 Summary

Phase 09 demonstrates how Tcl can combine file discovery, log analysis, report parsing, statistics, input validation, error handling, and report generation. Each script focuses on one task, while the final automation flow integrates several of the earlier concepts into a repeatable process.

The central lesson is to transform repetitive manual work into a predictable workflow: read inputs, parse relevant information, apply explicit checks, and generate a clear result. The supplied examples are educational starting points; production flows should validate their inputs and follow the actual tool and project requirements.

### Phase Completion Checklist

- ☐ Run `34_file_reporter.tcl` and verify the file count.
- ☐ Run `35_log_analyzer.tcl` and verify the four counters.
- ☐ Run `36_timing_report_parser.tcl` and explain the status decision.
- ☐ Populate `cells.rpt` with the documented sample data and verify cell statistics.
- ☐ Run `38_eda_automation_flow.tcl UART` and inspect the summary report.
- ☐ Explain how file handling, regular expressions, validation, and report writing work together.

**Next step:** Apply these Tcl automation patterns to your own EDA and HDL projects.

# Chapter 10

# Practical HDL Automation with Tcl

## 10.1 Introduction

Phase 10 applies Tcl automation to three Verilog projects: a D flip-flop, a 4-bit arithmetic logic unit (ALU), and a UART transmitter/receiver pair. In each project, the Tcl script coordinates compilation, simulation, log capture, and (for the ALU and UART projects) report generation. The testbenches create VCD waveform files for inspection with GTKWave.

The workflow uses Tcl, Icarus Verilog (`iverilog`), VVP (`vvp`), and GTKWave in an Ubuntu/WSL environment, as described in the supplied README.

## 10.2 Learning Objectives

By the end of this chapter, you should be able to:

- explain the role of RTL, a testbench, a simulator, and a Tcl runner;
- use Tcl's `exec` and `catch` to coordinate external tools;
- check compilation and simulation outcomes before proceeding;
- capture simulator output in log and report files;
- inspect generated VCD waveforms using GTKWave;
- compare the automation pattern across sequential, combinational, and multi-module designs;
- distinguish successful tool execution from functional correctness of the HDL.

## 10.3 Toolchain and Automation Architecture

### 10.3.1 Tools Used

| Tool | Role in the supplied workflow |
|---|---|
| `tclsh` | Interprets each project's Tcl automation script. |
| `iverilog` | Compiles Verilog RTL and its testbench into a simulation executable. |
| `vvp` | Executes the compiled simulation. |
| `GTKWave` | Displays signals recorded in a VCD waveform file. |
| `exec` | Tcl command used to run the external tools. |
| `catch` | Detects command errors so the script can print a failure message and stop. |

### 10.3.2 End-to-End Flow

**Verilog RTL + Testbench**
↓
**Tcl runner: compile with `iverilog`**
↓
**Check compilation result**
↓
**Run simulation with `vvp`**
↓
**Check simulation result and capture output**
↓
**Write log / report and generate VCD**
↓
**Inspect waveform with GTKWave**

The Tcl script coordinates commands and files. The testbench—not Tcl itself—creates the VCD waveform by calling Verilog system tasks such as `$dumpfile` and `$dumpvars`.

## 10.4 Project 1 — D Flip-Flop

### 10.4.1 Project Files

| File | Purpose |
|---|---|
| `D_flip_flop_behav.v` | Behavioral D flip-flop RTL. |
| `D_flip_flop_tb.v` | Testbench that drives D and clock. |
| `run.tcl` | Automates compilation, simulation, and log writing. |
| `dff_sim` | Compiled simulation executable. |
| `simulation.log` | Captured simulator output. |
| `dff.vcd` | Waveform data produced by the testbench. |
| `waveform.png` | Saved waveform image included in the project. |

### 10.4.2 RTL: `D_flip_flop_behav.v`

```
module D_flip_flop_behav(
    input d, clk,
    output reg q,
    output q_bar
);

    always @(posedge clk) begin
        q <= d;
    end
    assign q_bar = ~q;

endmodule
```

### 10.4.3 RTL Explanation

- `d` is the data input and `clk` is the clock.
- `always @(posedge clk)` updates the register on the rising edge.
- The nonblocking assignment `q <= d` schedules the output register to take the input value.
- `q_bar` is the complement of `q`.

### 10.4.4 Testbench: `D_flip_flop_tb.v`

```
module D_flip_flop_tb();
    reg d,clk;
    wire q,q_bar;

    D_flip_flop_behav uut(.d(d),.clk(clk),.q(q),.q_bar(q_bar));

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin
        d = 0; #10;
        d = 1; #10;
        d = 0; #10;
        d = 1; #10;
        d = 1; #10;
        d = 0; #10;
        d = 0; #10;
        $finish;
    end

    initial begin
      $dumpfile("dff.vcd");
      $dumpvars(0, D_flip_flop_tb);
    end
endmodule
```

The clock toggles every five simulation time units. The stimulus changes D every ten time units, and the simulation ends after the final delay. The VCD dump records the testbench's visible hierarchy and signals.

### 10.4.5 Tcl Runner: `run.tcl`

```
# Tcl automation for D flip-flop simulation

set rtl_file "D_flip_flop_behav.v"
set tb_file "D_flip_flop_tb.v"
set sim_file "dff_sim"
set log_file "simulation.log"

```

```tcl
puts "D Flip-Flop Automation"
puts "----------------------"

set status [catch {exec iverilog -o $sim_file $rtl_file $tb_file} result]

if {$status != 0} {
    puts "Compilation: FAIL"
    puts "Error: $result"
    exit 1
}
puts "Compilation: PASS"

set status [catch {exec vvp $sim_file} result]

if {$status != 0} {
    puts "Simulation: FAIL"
    puts "Error: $result"
    exit 1
}
puts "Simulation: PASS"

set file [open $log_file w]
puts $file $result
close $file

puts "Simulation log: $log_file"
```

### 10.4.6 How the Tcl Runner Works

1. File names are stored in variables to avoid repeating literal names in multiple commands.
2. `catch` runs the compiler command and stores its return code in `status`; the message or result is stored in `result`.
3. If compilation fails, the script prints an error and terminates with `exit 1`.
4. If compilation succeeds, the script runs the executable with `vvp`.
5. On successful simulation, the captured output is written to `simulation.log`.

### 10.4.7 Recorded Output

The source comments show the intended console summary:

```
D Flip-Flop Automation
----------------------
Compilation: PASS
Simulation: PASS
Simulation log: simulation.log
```

The supplied `simulation.log` contains:

```
VCD info: dumpfile dff.vcd opened for output.
D_flip_flop_tb.v:25: $finish called at 70 (1s)
```

This log indicates that the VCD dump was opened and the testbench reached `$finish`. A PASS from the Tcl runner indicates that the tool command completed without a reported command error; it does not by itself prove that every functional behavior has been checked by assertions.

### 10.4.8 Run and Inspect

From the project directory in Ubuntu/WSL:

```
tclsh run.tcl
gtkwave dff.vcd
```

The second command opens the generated waveform for visual inspection.

## 10.5 Project 2 — 4-Bit ALU

### 10.5.1 Project Files

The project includes `ALU_4_bit.v`, `ALU_4_bit_tb.v`, `run.tcl`, `alu_sim`, `simulation.log`, `sim_output.txt`, `alu_report.txt`, `alu.vcd`, and `waveform.png`.

### 10.5.2 RTL: `ALU_4_bit.v`

```
1  `timescale 1ns / 1ps
2
3  module ALU_4_bit(
4      input  [3:0] a,b,
5      input  [2:0] s,
6      output reg [3:0] y,
7      output reg carry,zero
8  );
9      always @(*) begin
10         carry = 0;
11         case (s)
12             3'b000 : {carry, y} = a + b;      // Add
13             3'b001 : {carry, y} = a - b;      // Sub
14             3'b010 : y = a & b;               // AND
15             3'b011 : y = a | b;               // OR
16             3'b100 : y = a ^ b;               // XOR
17             3'b101 : y = ~a;                  // NOT(on a)
18             3'b110 : y = a << b;              // Shift Left
19             3'b111 : y = a >> b;              // Shift Right
20             default: y = 4'b0000;             // Default
21         endcase
22         zero = (y == 4'b0000) ? 1'b1 : 1'b0;
23     end
24 endmodule
```

### 10.5.3 Operation Map

| Select | Operation | RTL expression |
|---|---|---|
| 000 | Addition | {carry,y}=a+b |
| 001 | Subtraction | {carry,y}=a-b |
| 010 | Bitwise AND | a & b |
| 011 | Bitwise OR | a \| b |
| 100 | Bitwise XOR | a ^ b |
| 101 | Bitwise NOT of A | ~a |
| 110 | Left shift A by B | a « b |
| 111 | Right shift A by B | a » b |

The `always @(*)` block describes combinational logic. The output `zero` is set when `y` equals zero. The RTL initializes `carry` to zero before the case statement; only the add/subtract assignments explicitly assign `carry` together with `y`.

### 10.5.4 Testbench: `ALU_4_bit_tb.v`

```
1  `timescale 1ns / 1ps
2  module ALU_4_bit_tb;
3      reg [3:0] a,b;
4      reg [2:0] s;
5      wire [3:0] y;
6      wire carry,zero;
7
8      ALU_4_bit uut (.a(a), .b(b),.s(s),.y(y),.carry(carry),.zero(zero));
9
10     initial begin
11         a = 4'b1010;
12         b = 4'b0101;
13
14         s = 3'b000; #10;
15         $display("t=%0t, s=%b, y=%b, carry=%b, zero=%b", $time, s, y, carry, zero)
       ;
16         s = 3'b001; #10;
17         $display("t=%0t, s=%b, y=%b, carry=%b, zero=%b", $time, s, y, carry, zero)
       ;
18         s = 3'b010; #10;
19         $display("t=%0t, s=%b, y=%b, carry=%b, zero=%b", $time, s, y, carry, zero)
       ;
20         s = 3'b011; #10;
21         $display("t=%0t, s=%b, y=%b, carry=%b, zero=%b", $time, s, y, carry, zero)
       ;
22         s = 3'b100; #10;
23         $display("t=%0t, s=%b, y=%b, carry=%b, zero=%b", $time, s, y, carry, zero)
       ;
24         s = 3'b101; #10;
25         $display("t=%0t, s=%b, y=%b, carry=%b, zero=%b", $time, s, y, carry, zero)
       ;
26         s = 3'b110; #10;
```

```
        $display("t=%0t, s=%b, y=%b, carry=%b, zero=%b", $time, s, y, carry, zero)
    ;
        s = 3'b111; #10;
        $display("t=%0t, s=%b, y=%b, carry=%b, zero=%b", $time, s, y, carry, zero)
    ;
        $finish;
    end

    initial begin
        $dumpfile("alu.vcd");
        $dumpvars(0, ALU_4_bit_tb);
    end
endmodule
```

The testbench fixes `a=1010` and `b=0101`, then steps through all eight select values. After each delay it prints the select code, output, carry, and zero flag.

### 10.5.5 Tcl Runner: `02_alu_4_bit/run.tcl`

```
set rtl_file "ALU_4_bit.v"
set tb_file "ALU_4_bit_tb.v"
set sim_file "alu_sim"
set log_file "simulation.log"
set report_file "alu_report.txt"

puts "4-bit ALU Automation"
puts "--------------------"

set status [catch {exec iverilog -o $sim_file $rtl_file $tb_file} result]
if {$status != 0} {
    puts "Compilation: FAIL"
    puts "Error: $result"
    exit 1
}
puts "Compilation: PASS"

set status [catch {exec vvp $sim_file} result]
if {$status != 0} {
    puts "Simulation: FAIL"
    puts "Error: $result"
    exit 1
}
puts "Simulation: PASS"

set file [open $log_file w]
puts $file $result
close $file
puts "Simulation log: $log_file"

set report [open $report_file w]
puts $report "4-bit ALU Simulation Report"
puts $report "--------------------------"
```

```
34 puts $report "Compilation: PASS"
35 puts $report "Simulation: PASS"
36 puts $report ""
37 puts $report "Simulation Output:"
38 puts $report $result
39 close $report
40 puts "Report generated: $report_file"
```

### 10.5.6 Recorded Simulation Results

The supplied `simulation.log` records:

```
VCD info: dumpfile alu.vcd opened for output.
t=10000, s=000, y=1111, carry=0, zero=0
t=20000, s=001, y=0101, carry=0, zero=0
t=30000, s=010, y=0000, carry=0, zero=1
t=40000, s=011, y=1111, carry=0, zero=0
t=50000, s=100, y=1111, carry=0, zero=0
t=60000, s=101, y=0101, carry=0, zero=0
t=70000, s=110, y=0000, carry=0, zero=1
t=80000, s=111, y=0000, carry=0, zero=1
ALU_4_bit_tb.v:40: $finish called at 80000 (1ps)
```

| Select | Output y | Carry | Zero | Operation |
|---|---|---|---|---|
| 000 | 1111 | 0 | 0 | Add |
| 001 | 0101 | 0 | 0 | Subtract |
| 010 | 0000 | 0 | 1 | AND |
| 011 | 1111 | 0 | 0 | OR |
| 100 | 1111 | 0 | 0 | XOR |
| 101 | 0101 | 0 | 0 | NOT A |
| 110 | 0000 | 0 | 1 | Shift left |
| 111 | 0000 | 0 | 1 | Shift right |

The output values above are taken from the supplied simulation log. They are presented as recorded rather than recalculated independently.

### 10.5.7 Report Generation

The ALU runner writes the captured simulator output to `simulation.log` and also creates `alu_report.txt`. The report contains a heading, compilation/simulation status lines, and the captured simulation output. This gives the project a compact record that can be reviewed without immediately reopening the simulator.

### 10.5.8 Run and Inspect

From the ALU project directory:

```
tclsh run.tcl
gtkwave alu.vcd
```

## 10.6 Project 3 — UART Transmitter and Receiver

### 10.6.1 Project Files

The UART project includes `UART_Tx.v`, `UART_Rx.v`, `UART_tb.v`, `run.tcl`, `uart_sim`, `simulation.log`, `sim_output.txt`, `uart_report.txt`, `uart.vcd`, and `waveform.png`.

### 10.6.2 Transmitter RTL: `UART_Tx.v`

```
`timescale 1ns / 1ps

module UART_Tx(
  input clk, reset, start,
  input [7:0] data,
  output reg tx
);
  reg [3:0] count;
  reg sending;

  always @(posedge clk or posedge reset) begin
    if (reset) begin
      tx <= 1;
      count <= 0;
      sending <= 0;
    end
    else if (start && !sending) begin
      sending <= 1;
      count <= 0;
    end
    else if (sending) begin
      case(count)
        0: tx <= 0;
        1: tx <= data[0];
        2: tx <= data[1];
        3: tx <= data[2];
        4: tx <= data[3];
        5: tx <= data[4];
        6: tx <= data[5];
        7: tx <= data[6];
        8: tx <= data[7];
        9: tx <= 1;
      endcase
      count <= count + 1;
      if (count == 9) sending <= 0;
    end
  end
endmodule
```

The transmitter uses an asynchronous active-high reset. Reset sets `tx` high (idle), clears the counter, and clears the `sending` flag. When `start` is asserted while not sending, the transmitter begins a frame. The case statement drives a start bit, eight data bits least-significant bit first,

and a stop bit.

### 10.6.3 Receiver RTL: `UART_Rx.v`

```
‘timescale 1ns / 1ps

module UART_Rx(
  input clk, reset, rx,
  output reg [7:0] data_out,
  output reg done
);
  reg [3:0] count;
  reg receiving;

  always @(posedge clk or posedge reset) begin
    if (reset) begin
      count <= 0;
      receiving <= 0;
      done <= 0;
    end
    else if (!receiving && rx == 0) begin
      receiving <= 1;
      count <= 0;
      done <= 0;
    end
    else if (receiving) begin
      count <= count + 1;
      case(count)
        1: data_out[0] <= rx;
        2: data_out[1] <= rx;
        3: data_out[2] <= rx;
        4: data_out[3] <= rx;
        5: data_out[4] <= rx;
        6: data_out[5] <= rx;
        7: data_out[6] <= rx;
        8: data_out[7] <= rx;
        9: begin done <= 1; receiving <= 0; end
      endcase
    end
  end
endmodule
```

The receiver waits for a low level to detect a start bit, then uses its counter to sample eight bits into `data_out`. At count 9 it asserts `done` and stops receiving. This is a simplified educational receiver; the supplied RTL does not include baud-rate timing, oversampling, or a synchronizer.

### 10.6.4 Testbench: `UART_tb.v`

```
‘timescale 1ns / 1ps

```

```
module UART_tb();
  reg clk = 0, reset, start;
  reg [7:0] data;
  wire tx;
  wire [7:0] rx_data;
  wire done;

  UART_Tx TX(clk, reset, start, data, tx);
  UART_Rx RX(clk, reset, tx, rx_data, done);

  always #5 clk = ~clk;

  initial begin
    reset = 1; start = 0; data = 8'h00;
    #10 reset = 0;

    #10 data = 8'hA5; start = 1; #10 start = 0;
    wait(done);
    $display("Received: %h", rx_data);
    #20 $finish;
  end

  initial begin
    $dumpfile("uart.vcd");
    $dumpvars(0, UART_tb);
  end
endmodule
```

The testbench connects the transmitter's `tx` output directly to the receiver's `rx` input. It attempts to send `8'hA5`, waits for the receiver's `done` signal, prints the received byte, and ends the simulation.

### 10.6.5 UART Tcl Runner: `run.tcl`

```
set tx_file "UART_Tx.v"
set rx_file "UART_Rx.v"
set tb_file "UART_tb.v"

set sim_file "uart_sim"
set log_file "simulation.log"
set report_file "uart_report.txt"

puts "UART TX/RX Automation"
puts "----------------------"

set status [catch {exec iverilog -o $sim_file $tx_file $rx_file $tb_file} result]
if {$status != 0} {
    puts "Compilation: FAIL"
    puts "Error: $result"
    exit 1
}
puts "Compilation: PASS"
```

```
19
20 set status [catch {exec vvp $sim_file} result]
21 if {$status != 0} {
22     puts "Simulation: FAIL"
23     puts "Error: $result"
24     exit 1
25 }
26 puts "Simulation: PASS"
27
28 set file [open $log_file w]
29 puts $file $result
30 close $file
31 puts "Simulation log: $log_file"
32
33 set report [open $report_file w]
34 puts $report "UART TX/RX Simulation Report"
35 puts $report "----------------------------"
36 puts $report "Compilation: PASS"
37 puts $report "Simulation: PASS"
38 puts $report ""
39 puts $report "Simulation Output:"
40 puts $report $result
41 close $report
42 puts "Report generated: $report_file"
```

### 10.6.6 Recorded Output and Important Functional Note

The supplied `simulation.log`, `sim_output.txt`, and `uart_report.txt` record:

```
VCD info: dumpfile uart.vcd opened for output.
Received: d2
UART_tb.v:23: $finish called at 165000 (1ps)
```

Although the testbench sets the transmit data to `8'hA5`, the provided simulation output says `Received: d2`. These values do not match. Therefore, the Tcl runner's `Simulation: PASS` only means that the simulator process returned successfully according to the script; it does not demonstrate that the UART transfer is functionally correct. This discrepancy should be investigated in the RTL/testbench timing and receiver sampling logic before considering the UART test passed.

### 10.6.7 Report Generation

The UART Tcl script writes simulator output to `simulation.log` and includes the same output in `uart_report.txt`. The report provides a convenient record of tool execution and the received byte. A stronger testbench should compare the received value against the expected byte and fail if they differ.

### 10.6.8 Run and Inspect

From the UART project directory:

```
tclsh run.tcl
gtkwave uart.vcd
```

## 10.7 Running the Projects in Ubuntu/WSL

### 10.7.1 Navigate to the Project Directory

The README specifies running each project's Tcl script from inside that project's directory. Replace the example base path with the real location of the repository.

```
cd /path/to/10_practical_hdl_automation/01_d_flip_flop
tclsh run.tcl
```

For the other projects:

```
cd /path/to/10_practical_hdl_automation/02_alu_4_bit
tclsh run.tcl
```

```
cd /path/to/10_practical_hdl_automation/03_uart
tclsh run.tcl
```

The Tcl scripts use project-local relative file names such as `D_flip_flop_behav.v` and `simulation.log`. Launching them from a different directory can cause the compiler or file operations to fail because the relative paths are resolved from the current working directory.

## 10.8 Understanding `exec` and `catch`

### 10.8.1 `exec`

The Tcl `exec` command starts an external process and returns its output. In these projects it launches `iverilog` for compilation and `vvp` for simulation.

### 10.8.2 `catch`

The form

```
1 set status [catch {exec iverilog -o $sim_file $rtl_file $tb_file} result]
```

runs the command inside `catch`. A return value of zero indicates that the script completed without a Tcl error; a nonzero value indicates an error was caught. The result variable receives the command result or error message. The runner uses this status to decide whether to print PASS or FAIL and whether to exit.

### 10.8.3 Important Limitation

The supplied scripts treat a successful `vvp` process exit as `Simulation: PASS`. They do not automatically establish functional correctness. A simulation can run to `$finish` while producing an incorrect output—as the UART recorded output demonstrates. Functional verification should include assertions or explicit expected-versus-actual comparisons and should return a failing status when a check fails.

## 10.9 Logs, Reports, and Waveforms

| Artifact | Purpose |
|---|---|
| `simulation.log` | Captures output returned by the simulator in each project. |
| `sim_output.txt` | Saved simulation output in the ALU and UART folders. |
| `alu_report.txt` | ALU execution summary plus simulator output. |
| `uart_report.txt` | UART execution summary plus simulator output. |
| `dff.vcd` | D flip-flop waveform data. |
| `alu.vcd` | ALU waveform data. |
| `uart.vcd` | UART waveform data. |
| `waveform.png` | Saved waveform image in each project directory. |

Logs are useful for textual evidence, reports collect key results in a consistent format, and waveforms help inspect signal transitions over time. They serve different purposes and should be used together when debugging.

## 10.10 Troubleshooting

| Problem | Checks |
|---|---|
| `iverilog` not found | Confirm Icarus Verilog is installed in the active Ubuntu/WSL environment and is on `PATH`. |
| RTL or testbench not found | Change into the correct project directory and verify exact filenames. |
| Compiled executable missing | Review compiler errors and confirm the compilation command succeeded. |
| No simulator output | Inspect `simulation.log`, the testbench's `$display` statements, and simulator exit status. |
| VCD file missing | Check `$dumpfile`, `$dumpvars`, and whether the testbench reaches the dump statements. |
| GTKWave cannot open VCD | Verify the file exists and contains waveform data. |
| UART byte mismatch | Compare expected and received data, then inspect start-bit detection, sampling points, counter timing, and waveform. |
| Runner reports PASS but behavior is wrong | Add functional checks; process success alone is not a correctness test. |

## 10.11 Practical Improvements

The projects are useful learning examples. The following improvements would make the automation more robust:

1. Add functional checks in testbenches, not only `$display` statements.
2. Make a testbench finish with a nonzero simulator exit status when a check fails.

3. Have the Tcl runner distinguish compile success, simulation-process success, and functional-test success.
4. Check that expected log, report, and VCD files were created.
5. Preserve useful diagnostics when compilation or simulation fails.
6. For the UART, investigate why the recorded result is `d2` when the stimulus byte is `a5`.
7. Keep scripts and reports reproducible by running from a known working directory.

## 10.12 Command Reference

| Command | Purpose |
|---|---|
| `tclsh run.tcl` | Runs the project's Tcl automation script. |
| `exec` | Invokes an external command from Tcl. |
| `catch` | Captures command errors and returns a status code. |
| `exit 1` | Stops the Tcl script with an error status. |
| `open` | Opens a log or report file. |
| `puts` | Prints console messages or writes to a file channel. |
| `close` | Closes a file channel. |
| `iverilog` | Compiles the Verilog design and testbench. |
| `vvp` | Runs the compiled simulation. |
| `gtkwave` | Opens a VCD file for waveform inspection. |
| `$dumpfile` | Selects the VCD output filename in the testbench. |
| `$dumpvars` | Selects hierarchy/signals to record in the VCD. |
| `$display` | Prints formatted simulation output. |
| `$finish` | Ends the simulation. |

## 10.13 Review Exercises

1. Explain the difference between the RTL, testbench, Tcl runner, and simulator.
2. Run the D flip-flop automation. Which messages appear in the console and which appear in `simulation.log`?
3. Explain how `catch` prevents the runner from continuing after a failed compilation.
4. Open `dff.vcd` in GTKWave and identify the clock, D, Q, and Q-bar signals.
5. List the eight ALU select codes and the operation associated with each.
6. Compare the ALU's console log with `alu_report.txt`. What information does the report add?
7. Run the UART flow and compare the transmitted stimulus byte with the recorded received byte.
8. Why is a successful simulator exit not sufficient evidence of functional correctness?

9. Propose a testbench check that fails if the received UART byte does not equal the expected byte.
10. Describe two ways to make the automation more robust when files or tools are missing.

## 10.14 Phase 10 Summary

Phase 10 connects Tcl scripting to practical Verilog simulation. The D flip-flop demonstrates the basic compile-and-run process, the 4-bit ALU demonstrates iteration across multiple operations and report generation, and the UART demonstrates a multi-module simulation with captured output and waveforms.

Across all three projects, Tcl coordinates external commands, checks tool errors, and organizes output files. A key engineering lesson is that automation status and functional correctness are separate: a command can execute successfully while the design produces an unexpected result. Functional checks and waveform review are therefore essential parts of a reliable HDL workflow.

### Phase Completion Checklist

☐ Run `01_d_flip_flop/run.tcl` from its project directory.

☐ Inspect `dff.vcd` using GTKWave.

☐ Run the ALU Tcl flow and review `simulation.log` and `alu_report.txt`.

☐ Inspect `alu.vcd` and relate the select code to the output.

☐ Run the UART flow and review `uart_report.txt`.

☐ Investigate the difference between transmitted `A5` and recorded received `d2`.

☐ Explain why `Compilation: PASS` and `Simulation: PASS` do not necessarily prove functional correctness.

**End of Phase 10:** Tcl automation applied to practical HDL simulation projects.

# Journey Completed

---

## Conclusion

This document has presented a practical learning journey from the fundamentals of Tcl scripting to its application in EDA workflows and HDL simulation automation.

Beginning with variables, expressions, control flow, and data structures, the journey progressed through procedures, string processing, file handling, pattern matching, error handling, automation utilities, EDA workflows, and practical Verilog simulation projects.

The central lesson is that effective automation is more than executing commands. It requires clear structure, reusable procedures, reliable file handling, meaningful error checks, readable reports, and careful verification of results.

The practical exercises and projects provide a foundation for continued learning and for developing more robust automation workflows. In particular, successful tool execution should always be distinguished from functional correctness. Logs, reports, testbench checks, and waveform analysis all contribute to confidence in the final result.

---

## Looking Ahead

Learning does not end with the completion of these phases. The next step is to apply these concepts to larger projects, improve existing scripts, automate repetitive engineering tasks, and continue exploring professional EDA tool flows.

Every script can be improved, every failure can teach something valuable, and every verified result builds stronger engineering judgment.

## Thank You

Bibhu Asish Panda

GitHub | LinkedIn

*Keep learning. Keep building. Keep verifying.*